

HỌC VIỆN KỸ THUẬT MẬT MÃ
KHOA ĐIỆN TỬ - VI MẠCH

-----□□&□□-----



BÁO CÁO MÔN HỌC
ĐỒ ÁN 2

NGHIÊN CỨU XÂY DỰNG HỆ THỐNG ĐIỀU KHIỂN VÀ
GIÁM SÁT THIẾT BỊ ĐIỆN TRONG NHÀ

Giảng viên hướng dẫn: ThS Chu Thị Ngọc Quỳnh

Sinh viên thực hiện: Hà Văn Đạt – DT060209

Dương Hải Đăng – DT060206

Nguyễn Mạnh Lâm – DT060231

Hà Nội – 2025

LỜI NÓI ĐẦU

Trong những năm gần đây, cùng với sự phát triển mạnh mẽ của khoa học và công nghệ, lĩnh vực Internet of Things (IoT) ngày càng được nghiên cứu và ứng dụng rộng rãi trong đời sống. Các hệ thống điều khiển và giám sát thiết bị điện thông minh góp phần nâng cao mức độ tiện nghi, cải thiện khả năng quản lý và tối ưu hóa việc sử dụng năng lượng trong các hộ gia đình, phù hợp với xu hướng phát triển của xã hội hiện đại.

Nhà thông minh là một trong những ứng dụng tiêu biểu của IoT, cho phép người dùng giám sát và điều khiển các thiết bị điện từ xa thông qua mạng Internet. Các hệ thống này thường kết hợp vi điều khiển, công nghệ truyền thông không dây và nền tảng quản lý trực tuyến. Đối với sinh viên ngành Điện tử Vi mạch, việc nghiên cứu và triển khai các hệ thống nhà thông minh giúp củng cố kiến thức chuyên môn và nâng cao khả năng thiết kế, tích hợp và ứng dụng công nghệ vào thực tiễn.

Xuất phát từ những yêu cầu đó, nhóm em lựa chọn thực hiện đề tài nghiên cứu xây dựng hệ thống điều khiển và giám sát thiết bị điện trong nhà trong khuôn khổ môn học Đồ án 2. Đề tài tập trung thiết kế một hệ thống điều khiển sử dụng vi điều khiển ESP32, cho phép điều khiển tám relay tương ứng với các thiết bị điện thông qua kết nối Internet, đồng thời tích hợp chức năng điều khiển bằng giọng nói nhằm nâng cao tính tiện lợi cho người sử dụng.

Trong quá trình thực hiện đề tài, nhóm em đã nghiên cứu và tham khảo các tài liệu kỹ thuật và hướng dẫn thiết kế liên quan đến lĩnh vực IoT và hệ thống nhà thông minh. Những tài liệu này đóng vai trò quan trọng trong việc định hướng thiết kế và đánh giá tính khả thi của hệ thống.

Cuối cùng, em xin bày tỏ lòng biết ơn sâu sắc tới Thạc sĩ Chu Thị Ngọc Quỳnh, người đã trực tiếp hướng dẫn và hỗ trợ trong suốt quá trình thực hiện đồ án. Sự tận tình của cô là động lực giúp chúng em hoàn thành đề tài đúng tiến độ và đạt được các mục tiêu đề ra.

MỤC LỤC

LỜI NÓI ĐẦU	
MỤC LỤC.....	
DANH MỤC TỪ VIẾT TẮT.....	
DANH MỤC HÌNH ẢNH	
DANH MỤC BẢNG BIỂU	
CHƯƠNG 1 : CƠ SỞ LÝ THUYẾT	1
1.1 Tổng quan về IoT và ứng dụng.....	1
1.2 Các cơ chế điều khiển các thiết bị điện trong nhà	2
1.2.1. Cơ chế điều khiển tại chỗ (Local Control).....	3
1.2.2. Cơ chế điều khiển từ xa qua mạng.....	4
1.2.3. Cơ chế điều khiển bằng giọng nói (Voice Control).....	5
1.3 Chuẩn giao tiếp sử dụng.....	5
1.3.1. Chuẩn kết nối không dây WiFi	6
1.3.2. Giao thức truyền thông MQTT	7
CHƯƠNG 2 : CÔNG NGHỆ THIẾT KẾ.....	8
2.1. Sơ đồ khối	8
2.2. Công nghệ phần cứng.....	9
2.2.1. Vi điều khiển trung tâm: ESP32 DevKit V1.....	9
2.2.2. Đóng cắt và cách ly công suất.....	10
2.2.3. Khối nguồn.....	13
2.3. Công nghệ phần mềm	14
2.3.1. Môi trường và nền tảng phát triển	14
2.3.2. Các giải pháp phần mềm tích hợp.....	15
2.3.2.1. Cấu hình WiFi động.....	16
2.3.2.2. Cập nhật firmware từ xa (OTA).....	18
2.3.2.3. Quản lý năng lượng.....	19
2.3.3. Các thư viện sử dụng chính.....	21
CHƯƠNG 3: THIẾT KẾ VÀ THI CÔNG	22
3.1. Sơ đồ mạch nguyên lý.....	22

3.2. Thiết kế mạch in PCB	22
3.3. Thiết kế phần mềm.....	23
3.4. Thi Công.....	26
3.4.1. Thi công phần cứng.....	26
3.4.2. Thi công phần mềm.....	27
3.4.2.1. Thiết lập môi trường và thư viện	27
3.4.2.2. Cấu hình hệ thống trên Arduino IoT Cloud.....	29
3.4.2.3. Cấu trúc chương trình và quản lý mã nguồn trên GitHub	31
3.4.2.4. Quy trình nạp và kiểm tra phần mềm	32
CHƯƠNG 4 : KẾT QUẢ VÀ ĐÁNH GIÁ	33
4.1. Mô hình thực tế	33
4.2. Kết quả đo kiểm	33
4.3. Đánh giá	36
4.4. Các lỗi phát sinh và hướng khắc phục	37
KẾT LUẬN	38
1. Ưu điểm.....	38
2. Nhược điểm.....	38
3. Phương hướng phát triển.....	39
PHỤ LỤC.....	41
File Main.c	41
FILE wifiConfig.h.....	44
FILE OTAUpdate.h.....	60

DANH MỤC TỪ VIẾT TẮT

AC	Dòng điện xoay chiều
Active	Trạng thái hoạt động
Cloud	Điện toán đám mây
Convert	Chuyển đổi
DC	Dòng điện một chiều
Firmware	Phần mềm nhúng điều khiển thiết bị
GND	Đất
IoT	Internet vạn vật
Local Control	Điều khiển tại chỗ
Mode	Chế độ hoạt động
Off	Trạng thái tắt
OTA	Cập nhật chương trình qua mạng (Over-The-Air)
PCB	Mạch in
Remote Control	Điều khiển từ xa
Smarthome	Nhà thông minh
SoC	Hệ thống trên chip (System on Chip)
VCC	Nguồn dương
Voice Control	Điều khiển bằng giọng nói
Web Server	Máy chủ web
WiFi	Mạng không dây

DANH MỤC HÌNH ẢNH

Hình 1.1: Tổng quan về IoT.....	1
Hình 1.2: Smart Home System	2
Hình 1.3: Nguyên lý hoạt động mạch điều khiển Relay.....	3
Hình 1.4: Sơ đồ luồng điều khiển qua nút nhấn	4
Hình 1.5: Sơ đồ luồng điều khiển từ xa qua internet	4
Hình 1.6: Sơ đồ luồng điều khiển từ xa qua giọng nói	5
Hình 1.7: Chế độ Station.....	6
Hình 1.8: Chế độ Access Point	7
Hình 2.1: Sơ đồ khối	8
Hình 2.2: ESP32 DEVKIT V1	9
Hình 2.2: Relay	11
Hình 2.3: PC817.....	11
Hình 2.4: Transitor 2N2222A	12
Hình 2.5: Diode 1N4007	12
Hình 2.6: Header 3 chân.....	13
Hình 2.7: Adapter 12V	13
Hình 2.8: Module LM2956	14
Hình 2.9: Arduino IDE 2.3.....	15
Hình 2.10: Arduino IoT Cloud.....	15
Hình 2.11: Giao diện cài đặt WiFi động của hệ thống	16
Hình 2.12: Các thành phần hoạt động của ESP32	20
Hình 3.1: Sơ đồ nguyên lý của hệ thống.....	22
Hình 3.2: Thiết kế PCB của hệ thống	23
Hình 3.3:Mạch 3D trên phần mềm	23
Hình 3.4: Lưu đồ giải thuật tổng quát của hệ thống	24
Hình 3.5: Chương trình xử lý các nút nhấn	25
Hình 3.6: Mạch khi chưa hàn linh kiện.....	26
Hình 3.7: Mặt trước.....	26
Hình 3.8: Mặt sau.....	27

Hình 3.9: Giao diện Arduino IDE 2.3 và cấu hình Board ESP32	29
Hình 3.10: Thing và khai báo các biến trên Arduino IoT Cloud	30
Hình 3.11: Dashboard điều khiển hệ thống trên Arduino IoT Cloud	30
Hình 3.12: Kho mã nguồn của dự án được quản lý trên nền tảng GitHub .	31
Hình 3.13: Theo dõi trạng thái hoạt động thông qua Serial Monitor	32
Hình 4.1: Kết quả phép thử điều khiển trực tiếp bằng nút nhấn.....	33
Hình 4.2: Giao diện điều khiển trên điện thoại.....	34
Hình 4.3: Màn hình điều khiển trên máy tính.....	34
Hình 4.4: Kết quả của phép thử điều khiển qua WiFi	35
Hình 4.5: Kiểm tra cài đặt wifi và ngày giờ.....	35
Hình 4.6: Kết quả phép thử cập nhật hệ thống qua OTA	36

DANH MỤC BẢNG BIỂU

Bảng 2.1 : Đặc điểm kỹ thuật của ESP32	10
Bảng 2.2: Chức năng cấu hình WiFi của hệ thống	17
Bảng 2.3: Chức năng cập nhật firmware OTA	18
Bảng 2.4: Các chế độ hoạt động của ESP32	19
Bảng 3.1: Danh sách thư viện và chức năng sử dụng trong hệ thống.....	28

CHƯƠNG 1 : CƠ SỞ LÝ THUYẾT

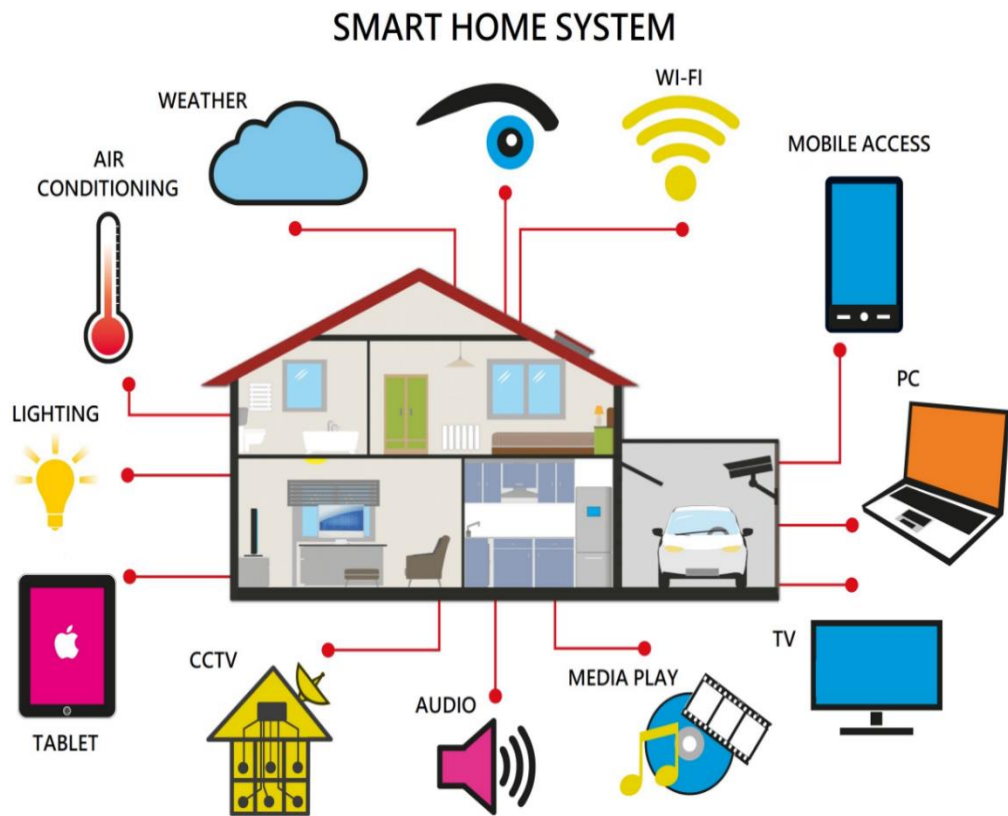
1.1 Tổng quan về IoT và ứng dụng

Internet vạn vật (IoT - Internet of Things) là một mạng lưới các thiết bị vật lý được tích hợp các bộ cảm biến, phần mềm và các công nghệ khác nhằm mục đích kết nối và trao đổi dữ liệu với các thiết bị và hệ thống khác qua Internet. Tương tự như cách hệ thần kinh điều khiển cơ thể con người, IoT cho phép các đồ vật vô tri (bóng đèn, quạt, cửa...) trở nên "thông minh", có khả năng thu thập dữ liệu và phản hồi lại các lệnh điều khiển từ con người hoặc tự động hóa dựa trên môi trường



Hình 1.1: Tổng quan về IoT

Nhà thông minh là một trong những ứng dụng tiêu biểu nhất của IoT, một ngôi nhà được trang bị hệ thống tự động tiên tiến để điều khiển đèn chiếu sáng, nhiệt độ, truyền thông đa phương tiện, an ninh, rèm cửa, cửa và nhiều tính năng khác. Mục tiêu cốt lõi của Smarthome không chỉ là sự tiện nghi (bật tắt từ xa) mà còn là quản lý năng lượng hiệu quả và đảm bảo an ninh. Trong một hệ thống Smarthome tiêu chuẩn, các thiết bị được kết nối với nhau qua một bộ điều khiển trung hoặc kết nối trực tiếp lên đám, cho phép chủ nhà giám sát ngôi nhà của mình thông qua điện thoại thông minh hoặc máy tính bảng ở bất kỳ đâu.



Hình 1.2: Smart Home System

Theo Nghị quyết 52-NQ/TW của Bộ Chính trị, Việt Nam đang đẩy mạnh chuyển đổi số, trong đó quản lý năng lượng hộ gia đình là một xu hướng tất yếu. Tuy nhiên, các thiết bị điện truyền thống thiếu khả năng giám sát và điều khiển từ xa, gây bất tiện và lãng phí điện năng.

Theo EVN, lãng phí điện sinh hoạt chủ yếu do thói quen sử dụng chưa hợp lý, đồng thời các công tắc cơ học truyền thống tiềm ẩn nguy cơ mất an toàn điện. Vì vậy, nhu cầu về hệ thống giám sát thời gian thực và điều khiển từ xa nhằm tiết kiệm năng lượng và đảm bảo an toàn là rất cần thiết.

1.2 Các cơ chế điều khiển các thiết bị điện trong nhà

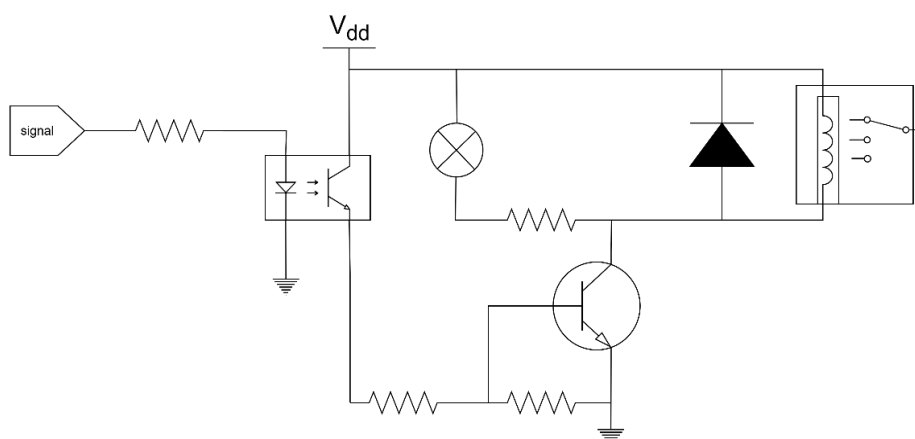
Để đảm bảo tính linh hoạt và tiện nghi cho người sử dụng, cũng như khả năng hoạt động ổn định trong mọi tình huống (kể cả khi mất kết nối mạng), hệ thống được thiết kế tích hợp đa dạng các cơ chế điều khiển. Các cơ chế này bao gồm: điều khiển tại chỗ thông qua nút nhấn vật lý và điều khiển thông minh từ xa thông qua nền tảng Internet of Things (IoT).

1.2.1. Cơ chế điều khiển tại chỗ (Local Control)

Điều khiển tại chỗ là phương thức cơ bản nhất, cho phép người dùng bật/tắt thiết bị trực tiếp thông qua các nút nhấn vật lý gắn trên thiết bị. Cơ chế này hoạt động độc lập với kết nối Internet, đảm bảo hệ thống luôn có thể vận hành ngay cả khi gặp sự cố về mạng.

Trong hệ thống điều khiển và giám sát thiết bị điện, khối điều khiển đóng vai trò cầu nối giữa vi điều khiển (hoạt động ở điện áp thấp 3.3V) và các thiết bị điện (hoạt động ở điện áp và công suất cao 220V). Do các chân GPIO của vi điều khiển chỉ có khả năng cung cấp dòng điện rất nhỏ, giải pháp phổ biến là sử dụng Relay kết hợp với các linh kiện bán dẫn trung gian để đảm bảo khả năng kích hoạt và an toàn cho mạch điều khiển.

Nguyên lý hoạt động của mạch điều khiển Relay được mô tả bằng hình dưới đây:



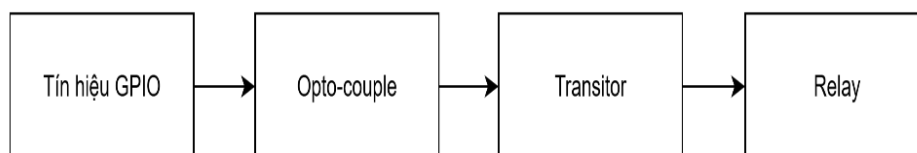
Hình 1.3: Nguyên lý hoạt động mạch điều khiển Relay

Quá trình điều khiển được thực hiện theo các bước sau: Khi hệ thống nhận lệnh điều khiển từ nút nhấn hoặc vi điều khiển, tín hiệu GPIO được đưa qua điện trở hạn dòng để kích hoạt Opto-coupler. Opto-coupler (PC817) đảm nhiệm chức năng cách ly quang giữa khối điều khiển điện áp thấp và khối công suất, giúp bảo vệ vi điều khiển khỏi nhiễu và các xung điện áp cao từ phía tải.

Tín hiệu sau cách ly được đưa đến tầng khuếch đại sử dụng Transistor (2N2222A), hoạt động như một công tắc điện tử nhằm cung cấp dòng điện đủ lớn

cho cuộn dây Relay. Khi Transistor dẫn, cuộn dây Relay được cấp điện làm đóng tiếp điểm, cho phép cấp nguồn cho tải; khi Transistor ngắt, Relay trở về trạng thái ban đầu và ngắt tải khỏi nguồn.

Để bảo vệ Transistor khỏi điện áp ngược do hiện tượng cảm ứng khi cuộn dây Relay ngắt, một Diode (1N4007) được mắc song song ngược với cuộn dây Relay nhằm triệt tiêu xung điện áp này.



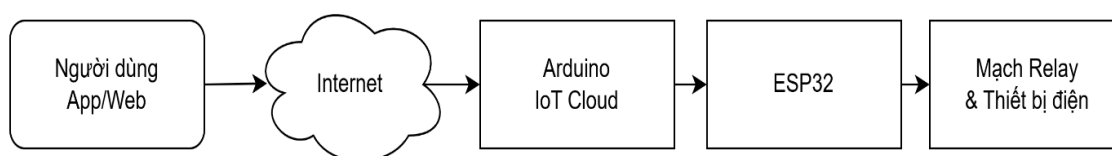
Hình 1.4: Sơ đồ luồng điều khiển qua nút nhấn

1.2.2. Cơ chế điều khiển từ xa qua mạng

Điều khiển từ xa qua Internet cho phép người dùng giám sát và điều khiển các thiết bị điện trong nhà thông qua nền tảng trực tuyến, không bị giới hạn bởi khoảng cách địa lý. Trong hệ thống này, vi điều khiển ESP32 đóng vai trò trung tâm, kết nối với mạng WiFi và trao đổi dữ liệu với nền tảng Arduino IoT Cloud theo thời gian thực. Các lệnh điều khiển từ người dùng được gửi thông qua giao diện web hoặc ứng dụng, sau đó được xử lý và chuyển thành tín hiệu điều khiển các relay tương ứng.

Cơ chế điều khiển này giúp nâng cao tính linh hoạt của hệ thống, cho phép người dùng quản lý thiết bị ở mọi lúc, mọi nơi. Đồng thời, việc sử dụng nền tảng Cloud giúp đảm bảo khả năng mở rộng, lưu trữ trạng thái và đồng bộ dữ liệu một cách ổn định.

Hình 1.x dưới đây mô tả luồng điều khiển từ xa của hệ thống:



Hình 1.5: Sơ đồ luồng điều khiển từ xa qua internet

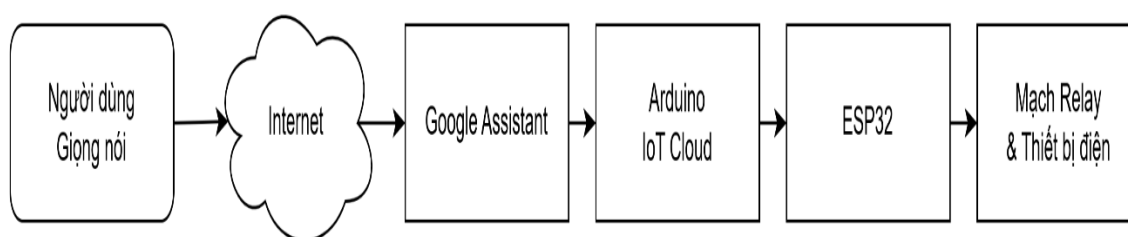
Độ trễ của hệ thống điều khiển phụ thuộc chủ yếu vào thời gian truyền dữ liệu qua mạng Internet và nền tảng Cloud. Trong điều kiện mạng ổn định, độ trễ nằm trong khoảng vài trăm mili giây, không ảnh hưởng đáng kể đến trải nghiệm điều khiển các thiết bị điện trong nhà.

1.2.3. Cơ chế điều khiển bằng giọng nói (Voice Control)

Điều khiển bằng giọng nói là một hình thức điều khiển hiện đại, giúp tăng tính tiện lợi và thân thiện cho người sử dụng. Trong hệ thống nhà thông minh, các lệnh giọng nói của người dùng được nhận diện và xử lý thông qua trợ lý ảo, sau đó chuyển thành lệnh điều khiển gửi tới nền tảng Arduino IoT Cloud. ESP32 tiếp nhận các lệnh này và thực hiện điều khiển các thiết bị điện tương ứng.

Hình thức điều khiển bằng giọng nói đặc biệt hữu ích trong các tình huống người dùng không thể thao tác trực tiếp bằng tay, đồng thời góp phần nâng cao trải nghiệm người dùng. Việc tích hợp điều khiển giọng nói cũng thể hiện tính hiện đại của hệ thống và phù hợp với xu hướng phát triển của các ứng dụng IoT hiện nay.

Hình dưới đây mô tả luồng điều khiển bằng giọng nói của hệ thống



Hình 1.6: Sơ đồ luồng điều khiển từ xa qua giọng nói

So với phương pháp điều khiển bằng nút nhấn hoặc giao diện web, điều khiển bằng giọng nói giúp giảm thao tác thủ công và nâng cao tính thân thiện với người dùng. Tuy nhiên, phương pháp này phụ thuộc vào kết nối Internet và độ chính xác của hệ thống nhận dạng giọng nói.

1.3 Chuẩn giao tiếp sử dụng

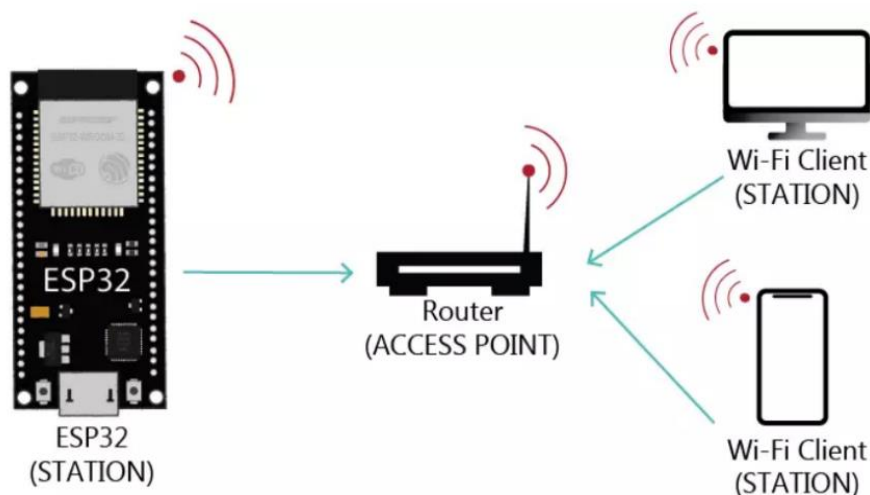
Để đảm bảo khả năng kết nối ổn định và truyền dữ liệu thời gian thực giữa thiết bị phân cứng và máy chủ đám mây, hệ thống sử dụng kết hợp chuẩn kết nối không dây WiFi và giao thức truyền thông MQTT. Đây là các chuẩn giao tiếp phổ biến, hiệu quả và phù hợp với các ứng dụng IoT dân dụng hiện nay.

1.3.1. Chuẩn kết nối không dây WiFi

Hệ thống sử dụng WiFi làm phương thức kết nối vật lý chính để truy cập Internet. Vi điều khiển ESP32 được tích hợp sẵn module WiFi hỗ trợ chuẩn IEEE 802.11 b/g/n, hoạt động ở băng tần 2.4 GHz. Việc sử dụng WiFi cho phép hệ thống dễ dàng tích hợp vào hạ tầng mạng gia đình sẵn có, không cần bổ sung bộ Gateway chuyên dụng như Zigbee hoặc LoRa.

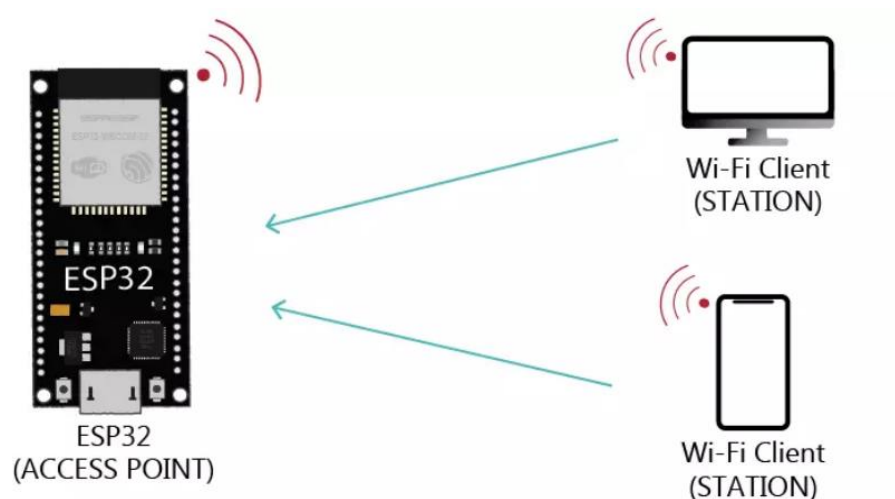
Trong quá trình vận hành, ESP32 được khai thác ở hai chế độ hoạt động chính là station và access point.

Chế độ Station (STA): Thiết bị đóng vai trò là Client kết nối vào Router WiFi gia đình để truy cập Internet. Ở chế độ này, ESP32 gửi dữ liệu trạng thái lên đám mây và nhận lệnh điều khiển từ người dùng



Hình 1.7: Chế độ Station

Chế độ Access Point (SoftAP): ESP32 phát ra một mạng WiFi riêng, được sử dụng trong giai đoạn khởi tạo hoặc khi mất kết nối Internet, cho phép người dùng cấu hình lại thông tin mạng trực tiếp mà không cần nạp lại chương trình.



Hình 1.8: Chế độ Access Point

1.3.2. Giao thức truyền thông MQTT

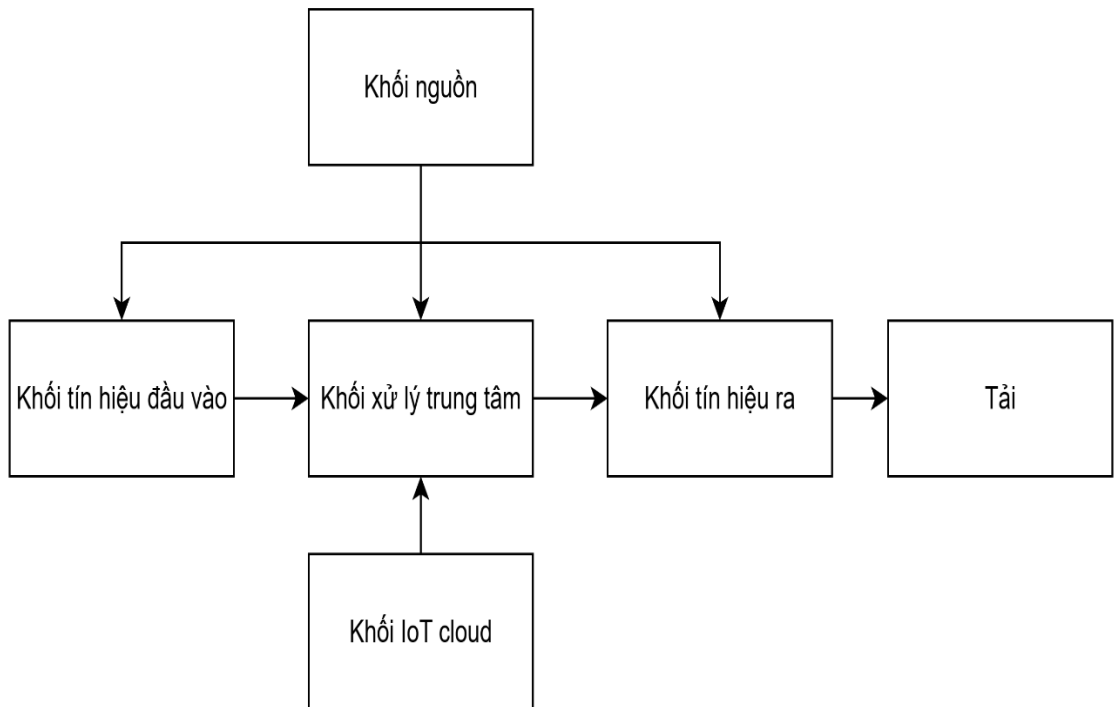
Trên nền tảng kết nối WiFi, hệ thống sử dụng giao thức MQTT để trao đổi dữ liệu với máy chủ Arduino IoT Cloud. MQTT là giao thức truyền thông theo mô hình Publish/Subscribe, được thiết kế tối ưu cho các thiết bị IoT có tài nguyên hạn chế và yêu cầu truyền dữ liệu hiệu quả.

Việc áp dụng giao thức MQTT giúp hệ thống đảm bảo khả năng truyền thông thời gian thực nhờ cơ chế duy trì kết nối liên tục với máy chủ, từ đó giảm độ trễ khi gửi lệnh điều khiển từ ứng dụng đến thiết bị. Bên cạnh đó, cấu trúc gói tin của MQTT nhỏ gọn, giúp tiết kiệm băng thông mạng và giảm tải xử lý cho vi điều khiển. Ngoài ra, dữ liệu trao đổi giữa thiết bị và máy chủ được truyền qua kết nối MQTT bảo mật, đảm bảo an toàn cho thông tin điều khiển và trạng thái thiết bị trong suốt quá trình vận hành.

CHƯƠNG 2 : CÔNG NGHỆ THIẾT KẾ

2.1. Sơ đồ khối

Hình dưới đây mô tả sơ đồ khối của hệ thống:



Hình 2.1: Sơ đồ khối

Khối nguồn: Cung cấp năng lượng cho toàn bộ hệ thống. Sử dụng Adapter để chuyển đổi điện áp lưới 220VAC xuống 12VDC, sau đó qua mạch hạ áp DC-DC để tạo ra mức 5V ổn định nuôi vi điều khiển.

Khối xử lý trung tâm: Đóng vai trò là "bộ não" của hệ thống, chịu trách nhiệm kết nối WiFi, xử lý logic điều khiển, giao tiếp với nền tảng đám mây và nhận tín hiệu từ người dùng.

Khối tín hiệu đầu vào: Gồm các nút nhấn vật lý cho phép người dùng điều khiển thiết bị tại chỗ (Local Control) khi không sử dụng điện thoại.

Khối IoT Cloud: Đảm nhiệm việc lưu trữ trạng thái, đồng bộ dữ liệu và cung cấp giao diện điều khiển từ xa qua Internet.

Khởi tín hiệu ra: Là tăng công suất thực thi lệnh đóng cắt. Khởi này sử dụng Relay kết hợp với Opto cách ly để điều khiển các thiết bị điện 220V một cách an toàn.

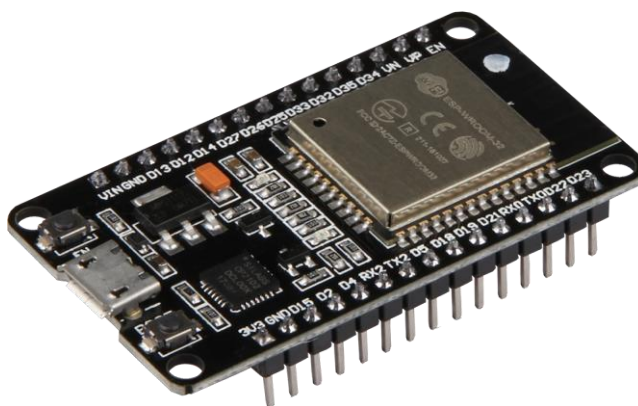
Khởi tải: bao gồm các thiết bị điện được điều khiển như đèn, quạt hoặc các thiết bị gia dụng khác. Các thiết bị này được cấp nguồn thông qua tiếp điểm của relay, cho phép hệ thống thực hiện chức năng đóng/cắt thiết bị theo lệnh điều khiển từ ESP32.

2.2. Công nghệ phần cứng

Hệ thống được xây dựng dựa trên các linh kiện bán dẫn và module chức năng phổ biến, đảm bảo tính kinh tế và khả năng thay thế dễ dàng.

2.2.1. Vi điều khiển trung tâm: ESP32 DevKit V1

ESP32 là dòng vi điều khiển SoC hiệu năng cao, giá thành thấp và tiêu thụ năng lượng thấp, tích hợp sẵn module WiFi chuẩn IEEE 802.11 b/g/n và Bluetooth hai chế độ, phù hợp cho các ứng dụng IoT. ESP32 sử dụng bộ vi xử lý Tensilica Xtensa LX6 với cấu hình lõi đơn hoặc lõi kép, đồng thời tích hợp đầy đủ các khối RF và mạch quản lý năng lượng trên cùng vi mạch. Vi điều khiển này do Espressif Systems phát triển, được sản xuất trên công nghệ 40 nm và là phiên bản kế thừa từ ESP8266.



Hình 2.2: ESP32 DEVKIT V1

Các đặc điểm kỹ thuật của ESP32 được trình bày thông qua bảng dưới đây:

Bảng 2.1 : Đặc điểm kỹ thuật của ESP32

Nhóm	Đặc điểm kỹ thuật
CPU	Xtensa LX6 32-bit, lõi đơn hoặc kép Tần số tối đa 240 MHz, hiệu năng lên đến 600 MIPS
Bộ nhớ trong	448KB ROM, 520KB SRAM
Kết nối không dây	Wi-Fi ,Bluetooth v4.2 BR/EDR và BLE
GPIO & Ngoại vi	34 chân GPIO, tích hợp bộ chuyển đổi ADC 12-bit với 18 kênh và DAC 2×8-bit, hỗ trợ các giao tiếp truyền thông như SPI, I ² C, I ² S và UART. Ngoài ra, vi điều khiển còn hỗ trợ CAN bus, Ethernet MAC, các chức năng PWM và LED PWM, bộ điều khiển hồng ngoại IR TX/RX, cùng cảm biến hiệu ứng Hall tích hợp.
Bảo mật	WPA/WPA2, Secure Boot, Mã hóa Flash, OTP 1024-bit, AES, SHA-2, RSA, ECC, RNG
Quản lý năng lượng	5 chế độ: Active, Modem-sleep, Light-sleep, Deep-sleep, Hibernation

2.2.2. Đóng cắt và cách ly công suất

Relay 12V - 10A: Nguồn chính của hệ thống là adapter AC–DC 12 V, LM2596 tạo ra 5 V cấp cho ESP32. Sử dụng relay cuộn dây 12 V tận dụng trực tiếp nguồn 12 V, giảm linh kiện, đơn giản hóa thiết kế và nâng cao độ ổn định. Relay có dòng chịu tải 10 A, phù hợp điều khiển các thiết bị dân dụng như đèn

chiếu sáng (0,3–0,5 A), quạt điện (0,5–1 A) hoặc thiết bị công suất trung bình (<5 A), đảm bảo hệ số an toàn cao và tuổi thọ lâu dài.



Hình 2.2: Relay

Opto-coupler PC817: PC817 cách ly điện hoàn toàn giữa khối điều khiển và cuộn dây relay 12 V, giảm nhiễu, xung áp và dòng ngược tác động lên ESP32. Điện áp cách ly lên đến 5 kV, dòng LED đầu vào vài mA, phù hợp trực tiếp với GPIO ESP32 qua điện trở hạn dòng. Hệ số truyền dòng (CTR) đủ lớn để kích hoạt transistor phía sau, đảm bảo relay đóng cắt dứt khoát.

Điện trở hạn dòng cho LED được tính theo công thức:

$$R \geq \frac{12 - 1,9}{20 \times 10^{-3}} = 505 \, \Omega$$

Vậy ta chọn giá trị điện trở này là 1k Ω .



Hình 2.3: PC817

Transistor 2N2222A: Transistor 2N2222A được chọn dựa trên $V_{CE} = 40$ V, $I_{Cmax} = 800$ mA, đáp ứng dòng cuộn dây relay 80 mA. Điện trở Base–Emitter 10 k Ω bảo vệ transistor khỏi nhiễu.

Điều kiện bão hòa của transistor:

$$I_{B(sat)} = \frac{I_C}{\beta_{min}} = \frac{80 \text{ mA}}{75} \approx 1,07 \text{ mA}$$

Dòng thực tế qua Base:

$$I_B = \frac{V_{OH} - V_{BE}}{R_B} = \frac{3,3 - 0,7}{1000} = 0,26 \text{ mA}$$

Hệ số vượt dòng $\approx 2,43$, đảm bảo transistor dẫn bão hòa an toàn và dòng vào GPIO ESP32 $\leq 12 \text{ mA}$.

Khi transistor dẫn, dòng Irelay chạy qua cuộn dây ổn định, tích trữ năng lượng dưới dạng từ trường:

$$E = \frac{1}{2} LI^2$$



Hình 2.4: Transistor 2N2222A

Diode 1N4007: Mắc ngược chiều với cuộn dây Relay để triệt tiêu dòng điện cảm ứng (dòng xả) khi ngắt mạch, bảo vệ Transistor .

Khi transistor tắt, cuộn dây relay tạo xung điện áp ngược (Back EMF) theo định luật Len-xơ:

$$V = L \frac{di}{dt}$$

Diode 1N4007 mắc song song với cuộn dây (Cathode +12 V, Anode nối cực C transistor) tạo vòng kín cho dòng cảm ứng, tiêu tán năng lượng, bảo vệ transistor. Điện áp cực C transistor:

$$V_{Cmax} = V_{CC} + V_D = 12 + 0,7 = 12,7 \text{ V} < V_{CEmax} = 40 \text{ V}$$

1N4007 có $U_{max} = 1000 \text{ V}$ và dòng liên tục 10 A, phù hợp và giá thành thấp.



Hình 2.5: Diode 1N4007

Ngoài ra để thuận tiện kết nối với tải, nhóm còn sử dụng thêm các header 3 chân:



Hình 2.6: Header 3 chân

Kết luận: Các linh kiện được lựa chọn đáp ứng yêu cầu dòng, áp và đảm bảo hệ số an toàn.

2.2.3. Khối nguồn

Adapter AC-DC 12V: Chuyển đổi điện áp 220VAC sang 12VDC, đảm bảo cách ly an toàn ngay từ đầu vào .



Hình 2.7: Adapter 12V

Xét bài toán năng lượng tổng thể cho hệ thống bao gồm 1 module ESP32 và 8 kênh Relay hoạt động đồng thời:

Dòng tiêu thụ cực đại của ESP32 (khi bật WiFi và truyền dữ liệu) : $I_{ESP} \approx 260mA$

Dòng tiêu thụ của một Relay 12V (bao gồm dòng kích cuộn dây và đèn LED báo hiệu): $I_{Relay} \approx 80mA$. Với 8 Relay, tổng dòng là:

$$I_{RelayTotal} = 8 \times 80mA = 640mA.$$

Dòng tiêu thụ của các đèn LED báo trạng thái và Optocoupler: $I_{Signal} \approx 320mA$. Tổng dòng tiêu thụ lý thuyết tại tải:

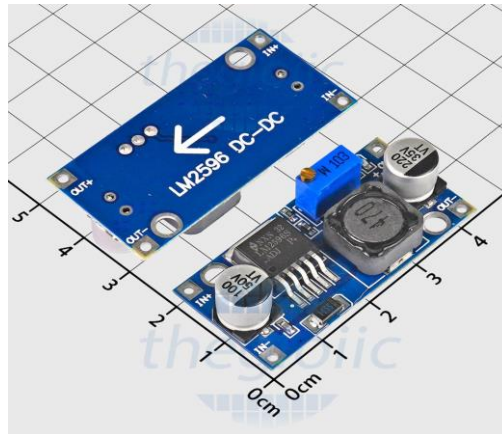
$$I_{Total} = I_{ESP} + I_{RelayTotal} + I_{Signal} \approx 260 + 640 + 320 = 1220mA$$

Tuy nhiên, trong kỹ thuật thiết kế nguồn, hệ số an toàn (Safety Factor) thường được chọn là 2

$$I_{Source} \geq I_{Total} \times 2 = 1220 \times 2 \approx 2.44A$$

Đáp ứng dòng khởi động tức thời (Inrush current) của các cuộn cảm trong Relay và tụ điện. Tránh hiện tượng sụt áp khi ESP32 thực hiện các tác vụ nặng (Burst transmission).

Module hạ áp LM2596: Mạch giảm áp xung (Buck Converter) hiệu suất cao, chuyển đổi 12V xuống 5V để cấp nguồn cho ESP32. Module này hoạt động ổn định và tỏa nhiệt ít hơn so với các IC ổn áp tuyến tính truyền thống (như 7805).



Hình 2.8: Module LM2596

2.3. Công nghệ phần mềm

Phần mềm hệ thống được phát triển để đảm bảo tính ổn định, khả năng phản hồi nhanh và trải nghiệm người dùng thuận tiện.

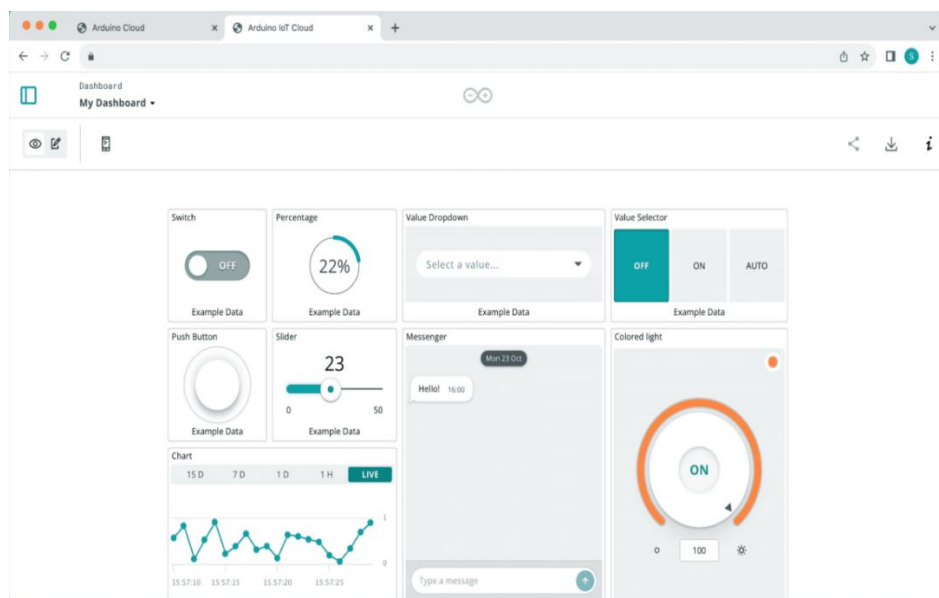
2.3.1. Môi trường và nền tảng phát triển

Arduino IDE 2.3: Công cụ lập trình chính, hỗ trợ biên dịch mã nguồn C/C++ và nạp chương trình cho ESP32. Sử dụng hệ thống thư viện phong phú để xử lý kết nối mạng và phần cứng.



Hình 2.9: Arduino IDE 2.3

Arduino IoT Cloud: Nền tảng đám mây cung cấp dịch vụ Broker MQTT, lưu trữ biến trạng thái (Thing Properties) và thiết kế giao diện Dashboard (Web/Mobile App) mà không cần tự xây dựng Server.



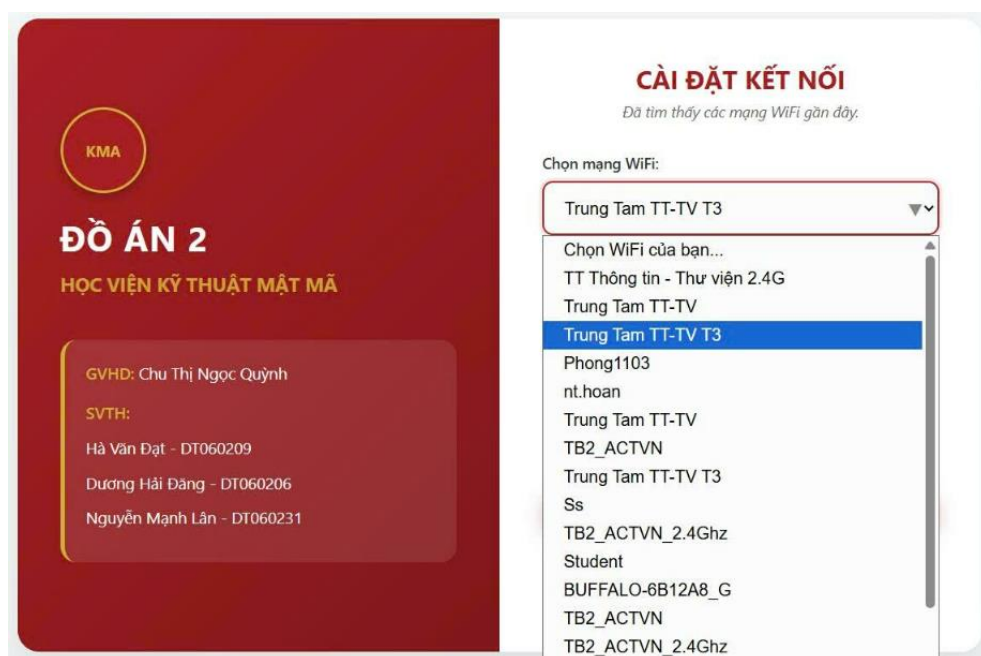
Hình 2.10: Arduino IoT Cloud

2.3.2. Các giải pháp phần mềm tích hợp

Để khắc phục các hạn chế của hệ thống IoT cơ bản, phần mềm được thiết kế tích hợp các chức năng nâng cao.

2.3.2.1. Cấu hình WiFi động

Hiện nay, Arduino IoT Cloud cung cấp thư viện `thingProperties.h` cho phép thiết bị IoT kết nối tới nền tảng Cloud và đồng bộ dữ liệu theo thời gian thực một cách thuận tiện. Tuy nhiên, thư viện này chỉ hỗ trợ cơ chế kết nối WiFi tĩnh, trong đó thông tin tên mạng và mật khẩu WiFi được khai báo cố định trong chương trình. Khi môi trường mạng thay đổi hoặc thiết bị được di chuyển sang vị trí sử dụng khác, người dùng bắt buộc phải nạp lại chương trình cho vi điều khiển. Điều này gây ra nhiều bất tiện trong quá trình triển khai thực tế, đặc biệt đối với các hệ thống IoT được lắp đặt cố định, cần vận hành lâu dài và thường xuyên thay đổi hạ tầng mạng.



Hình 2.11: Giao diện cài đặt WiFi động của hệ thống

Để khắc phục hạn chế của Arduino IoT cloud, nhóm đã thiết kế và xây dựng mô-đun cấu hình WiFi độc lập (`wifiConfig.h`), cho phép: Cấu hình WiFi trực tiếp qua trình duyệt web cho phép không cần nạp lại chương trình khi đổi mạng, lưu thông tin WiFi vào bộ nhớ eeprom và có thể khôi phục và cấu hình lại khi cần.

Giải pháp này giúp hệ thống linh hoạt hơn, dễ triển khai thực tế, đồng thời tách biệt chức năng cấu hình mạng khỏi logic điều khiển chính. Đồng thời nâng cao tính hiệu quả của hệ thống.

Bảng 2.2: Chức năng cấu hình WiFi của hệ thống

Nội dung	Mô tả
Lưu thông tin WiFi	SSID và mật khẩu WiFi được lưu trong bộ nhớ không mất dữ liệu, giúp thiết bị tự động kết nối lại sau khi khởi động.
Chế độ kết nối STA	Khi đã có thông tin WiFi hợp lệ, ESP32 hoạt động ở chế độ Station và kết nối vào mạng WiFi hiện có.
Chế độ cấu hình AP	Khi không kết nối được WiFi hoặc chưa có thông tin mạng, ESP32 tự động chuyển sang chế độ Access Point
Tên và địa chỉ truy cập	Thiết bị phát WiFi với SSID riêng và cung cấp giao diện cấu hình qua địa chỉ IP mặc định 192.168.4.1.
Giao diện Web	Web server tích hợp cho phép người dùng nhập và lưu thông tin WiFi trực tiếp trên trình duyệt.
Cơ chế xác thực truy cập	Để bảo vệ cấu hình hệ thống, giao diện web được bổ sung cơ chế xác thực bằng mật khẩu, giúp hạn chế truy cập trái phép và nâng cao mức độ an toàn cho thiết bị.
Áp dụng cấu hình mới	Dữ liệu mới được ghi vào EEPROM và thiết bị tự động khởi động lại để áp dụng cấu hình mới, đồng thời tiến hành kết nối tới mạng WiFi vừa được thiết lập.
Tự động chuyển chế độ	Sau khi cấu hình thành công, thiết bị tự động thoát chế độ AP và chuyển sang chế độ STA.
Duy trì hoạt động	Hàm xử lý WiFi được gọi liên tục trong vòng lặp chính để đảm bảo web server và trạng thái kết nối luôn ổn định.
Reset WiFi	Nút nhấn cứng cho phép người dùng đặt lại wifi
Tích hợp với chương trình chính	Mô-đun wifiConfig.h được thiết kế độc lập và tích hợp vào chương trình chính thông qua việc gọi hàm khởi tạo trong setup() và hàm duy trì trong loop(), đảm bảo không ảnh hưởng đến logic điều khiển chính của hệ thống.

Việc bổ sung mô-đun cấu hình WiFi động giúp khắc phục hoàn toàn hạn chế kết nối WiFi tĩnh của Arduino IoT Cloud, đồng thời nâng cao tính linh hoạt, khả năng mở rộng và tính ứng dụng thực tế của hệ thống.

2.3.2.2. Cập nhật firmware từ xa (OTA)

Hệ thống tích hợp khả năng tự động kiểm tra và tải bản cập nhật phần mềm mới từ kho lưu trữ GitHub, giúp bảo trì và nâng cấp tính năng thiết bị từ xa

Bằng cách triển khai module OTA Update độc lập, cho phép: Kiểm tra firmware mới từ server (GitHub), tải firmware nhị phân qua internet, ghi firmware trực tiếp vào ESP32, tự động khởi động lại và áp dụng firmware mới.

Bảng 2.3: Chức năng cập nhật firmware OTA

Nội dung	Mô tả
Kiểm tra phiên bản	ESP32 lấy phiên bản firmware mới nhất từ máy chủ và so sánh với phiên bản hiện tại của hệ thống.
So sánh phiên bản	Nếu có phiên bản mới hơn ESP sẽ tự động cập nhật
Điều kiện cập nhật	Quá trình OTA chỉ được thực hiện khi thiết bị có kết nối WiFi ổn định.
Nguồn firmware	Firmware được lưu trữ trên máy chủ Internet và cho phép truy cập thông qua giao thức HTTP hoặc HTTPS.
Tải và ghi firmware	Firmware mới được tải về và ghi trực tiếp vào bộ nhớ Flash của ESP32 bằng thư viện Update.
Giám sát quá trình	Hệ thống theo dõi dung lượng dữ liệu nhận được và kiểm soát lỗi trong suốt quá trình cập nhật.
Hoàn tất OTA	Sau khi cập nhật thành công, thiết bị tự động khởi động lại để chạy firmware mới.

Việc tích hợp cơ chế OTA Update cho phép hệ thống tự động kiểm tra, tải và cập nhật firmware từ xa, góp phần nâng cao khả năng bảo trì, mở rộng và tính ứng dụng thực tế của hệ thống IoT.

2.3.2.3. Quản lý năng lượng

Vấn đề tiêu thụ năng lượng là bài toán cốt lõi trong thiết kế thiết bị IoT. ESP32 cung cấp khả năng quản lý năng lượng linh hoạt thông qua 5 chế độ hoạt động chính, cho phép tối ưu hóa dòng tiêu thụ tùy theo yêu cầu xử lý tại từng thời điểm.

Bảng sau mô tả chi tiết các chế độ hoạt động của ESP:

Bảng 2.4: Các chế độ hoạt động của ESP32

Chế độ nguồn	Mô tả			Dòng tiêu thụ
Modem-sleep	CPU vẫn được cấp nguồn hoạt động	240 MHz	Chíp lõi kép	30mA-68mA
			Chíp lõi đơn	N/A
		160 MHz	Chíp lõi kép	27mA-44mA
			Chíp lõi đơn	27mA-34mA
		80 MHz	Chip lõi kép	20mA - 31mA
			Chip lõi đơn	20mA - 25mA
Light-sleep				0.8mA
Deep-sleep	Bộ đồng xử lý ULP (Ultra Low Power) được cấp nguồn			150μA
	Mô hình giám sát cảm biến qua ULP			100μA(chu kì làm việc 1%)
	Bộ định thời RTC + Bộ nhớ RTC			10μA
Hibernation	Chỉ bộ định thời (RTC timer) hoạt động			5μA
Power off	Chân CHIP_PU được đặt xuống mức thấp (Lower level), chip bị tắt nguồn hoàn toàn			1μA

Hình dưới đây mô tả các thành phần hoạt động khi ESP32 hoạt động :



Hình 2.12: Các thành phần hoạt động của ESP32

Active Mode (Chế độ tích cực): Trong chế độ này, tất cả các thành phần của chip bao gồm CPU, module WiFi, Bluetooth và các thiết bị ngoại vi đều được cấp điện. Dòng tiêu thụ dao động từ 160mA đến 260mA. Đây là chế độ mặc định khi hệ thống xử lý dữ liệu hoặc truyền nhận tin qua mạng.

Modem-sleep Mode: CPU vẫn hoạt động nhưng các module vô tuyến (WiFi/Bluetooth/Radio) bị tắt. Chế độ này giúp duy trì kết nối với Router (bằng cách bật tắt WiFi định kỳ theo chuẩn DTIM Beacon) nhưng giảm đáng kể năng lượng tiêu thụ xuống khoảng 3mA đến 20mA.

Light-sleep Mode: CPU tạm dừng hoạt động (clock bị ngắt), nhưng trạng thái của bộ nhớ RAM được duy trì. Hệ thống có thể được đánh thức nhanh chóng. Dòng tiêu thụ xấp xỉ 0.8mA.

Deep-sleep Mode: CPU và hầu hết các thiết bị ngoại vi bị tắt nguồn hoàn toàn, chỉ duy trì bộ xử lý ULP (Ultra Low Power) và bộ định thời RTC. Dữ liệu trên RAM bị mất (trừ vùng nhớ RTC Fast Memory). Dòng tiêu thụ cực thấp, chỉ khoảng 10μA đến 150μA.

Hibernation Mode: Chế độ ngủ đông tắt hầu hết mọi thứ, chỉ giữ lại một bộ định thời RTC duy nhất để đánh thức chip. Dòng tiêu thụ thấp nhất, khoảng 5μA.

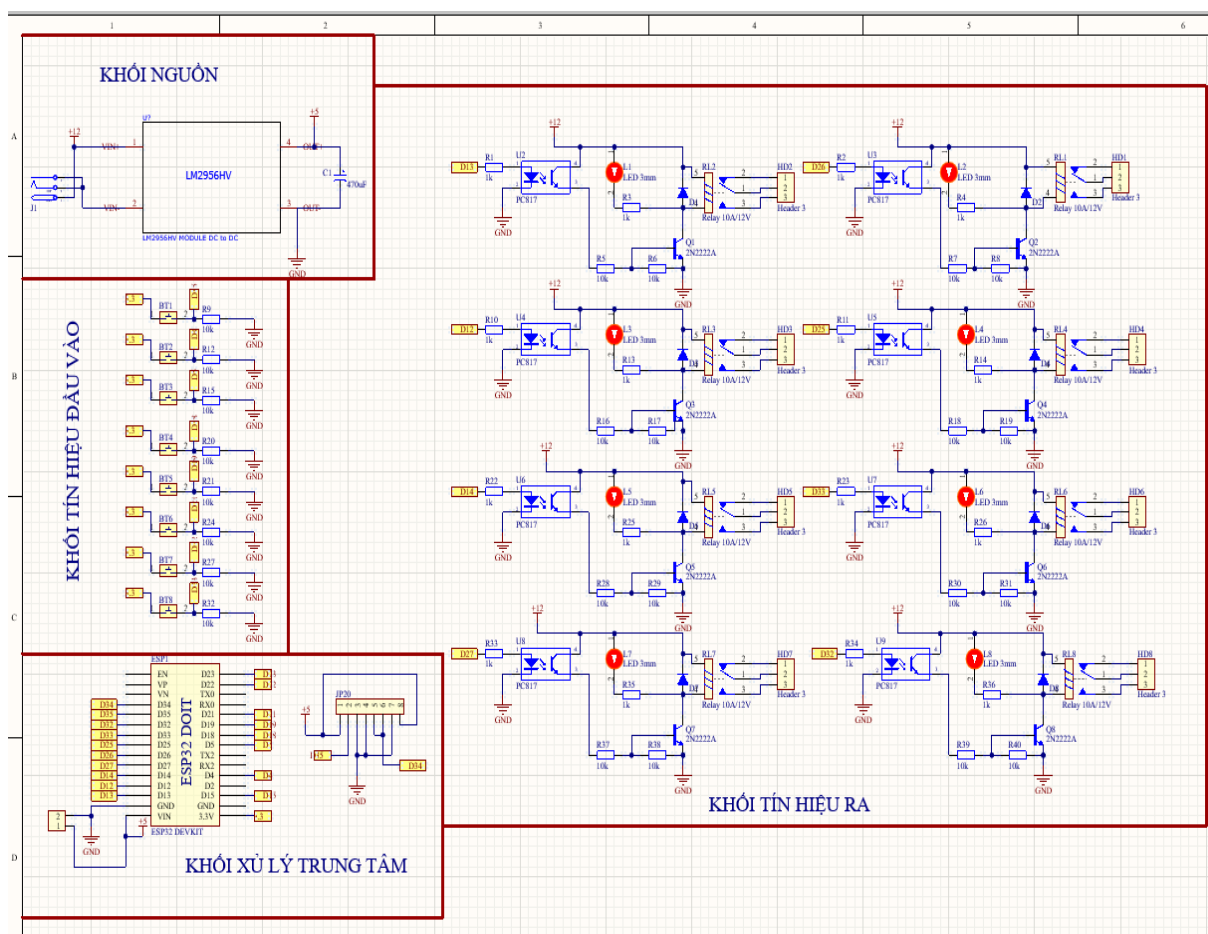
2.3.3. Các thư viện sử dụng chính

Để hiện thực hóa các chức năng của hệ thống, chương trình sử dụng một số thư viện phần mềm quan trọng. Thư viện `thingProperties.h` đảm nhiệm việc quản lý kết nối giữa thiết bị và nền tảng Arduino IoT Cloud, đồng thời hỗ trợ đồng bộ dữ liệu theo thời gian thực. Thư viện `WiFi.h` và `WebServer.h` được sử dụng để thiết lập kết nối mạng không dây và xây dựng giao diện Web Server nội bộ, phục vụ cho cấu hình WiFi động và tương tác với người dùng. Bên cạnh đó, chức năng cập nhật firmware từ xa (OTA) được triển khai thông qua thư viện `OTAUpdate.h`, cho phép thiết bị tải và ghi firmware mới vào bộ nhớ chương trình một cách an toàn. Việc kết hợp các thư viện tiêu chuẩn của Arduino và ESP32 cùng với các module tự xây dựng giúp hệ thống hoạt động ổn định, linh hoạt và đáp ứng tốt khả năng mở rộng trong tương lai.

CHƯƠNG 3: THIẾT KẾ VÀ THI CÔNG

3.1. Sơ đồ mạch nguyên lý

Sơ đồ nguyên lý toàn hệ thống được thiết kế gồm các khối chức năng chính liên kết với nhau. Khối nguồn sử dụng module LM2596 để hạ áp và cung cấp điện áp 5 V cho vi điều khiển ESP32. Ở khối đầu vào, ESP32 kết nối với 8 nút nhấn, mỗi nút được mắc điện trở kéo xuống nhằm đảm bảo trạng thái logic ổn định. Khối đầu ra gồm 8 chân GPIO của ESP32 điều khiển các module cách ly quang PC817 và mạch kích relay, giúp cách ly và bảo vệ mạch điều khiển khi đóng cắt tải bên ngoài.



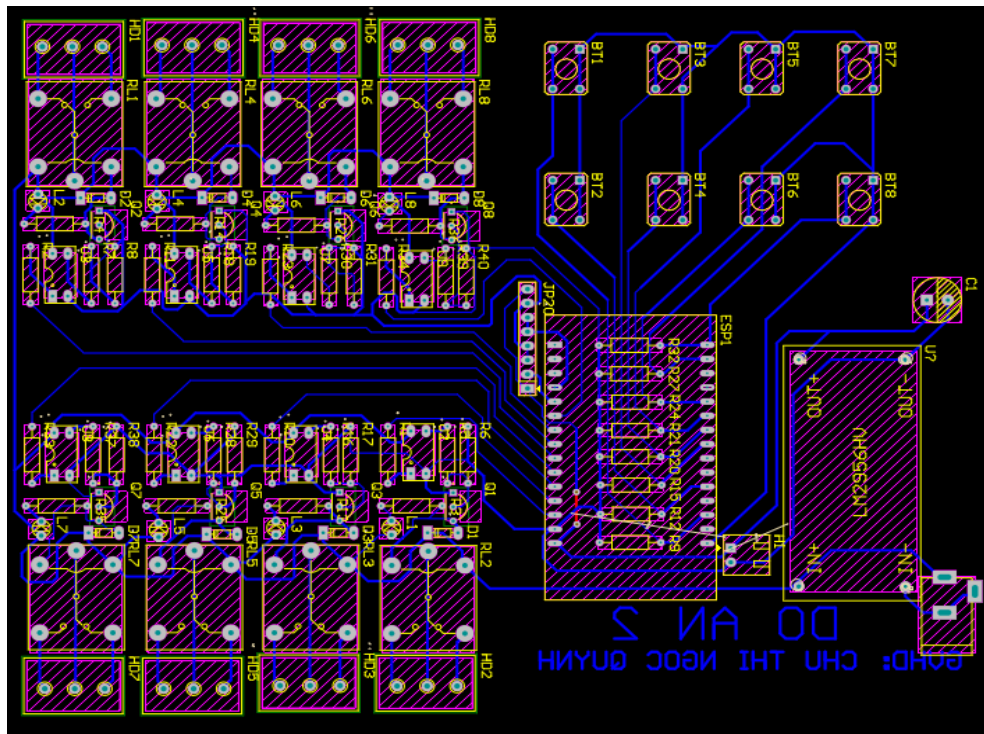
Hình 3.1: Sơ đồ nguyên lý của hệ thống

3.2. Thiết kế mạch in PCB

Mạch in được thiết kế một lớp, trong đó các đường mạch động lực điều khiển relay có độ rộng lớn hơn 60 mil và được phủ thiếc để chịu dòng cao. Phần mạch điều khiển 3,3 V được bố trí tách biệt với khu vực công suất 220 V nhằm giảm

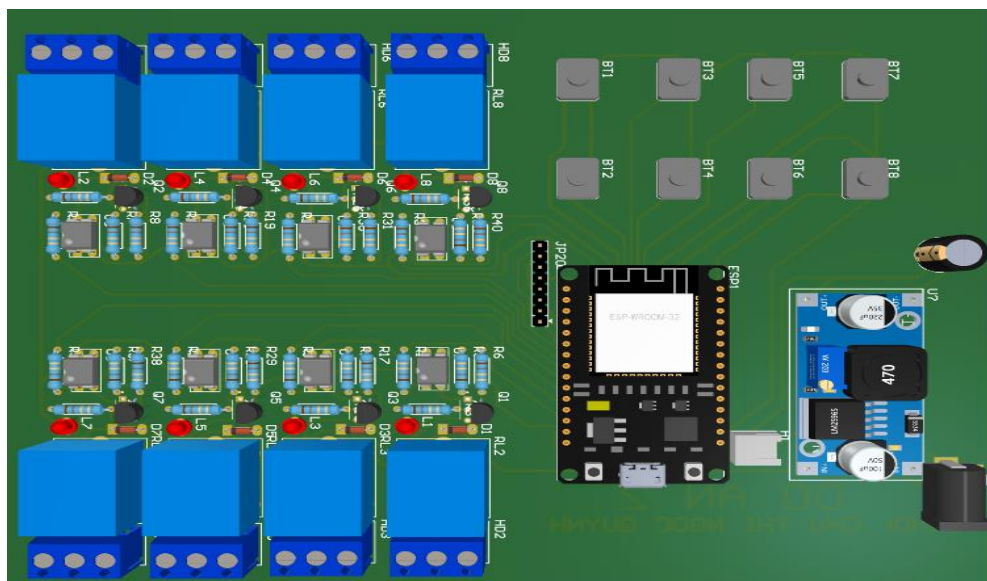
hiệu và đảm bảo an toàn. ESP32 được gắn qua header để thuận tiện cho việc tháo lắp và nạp chương trình

Hình dưới đây là thiết kế PCB của hệ thống :



Hình 3.2: Thiết kế PCB của hệ thống

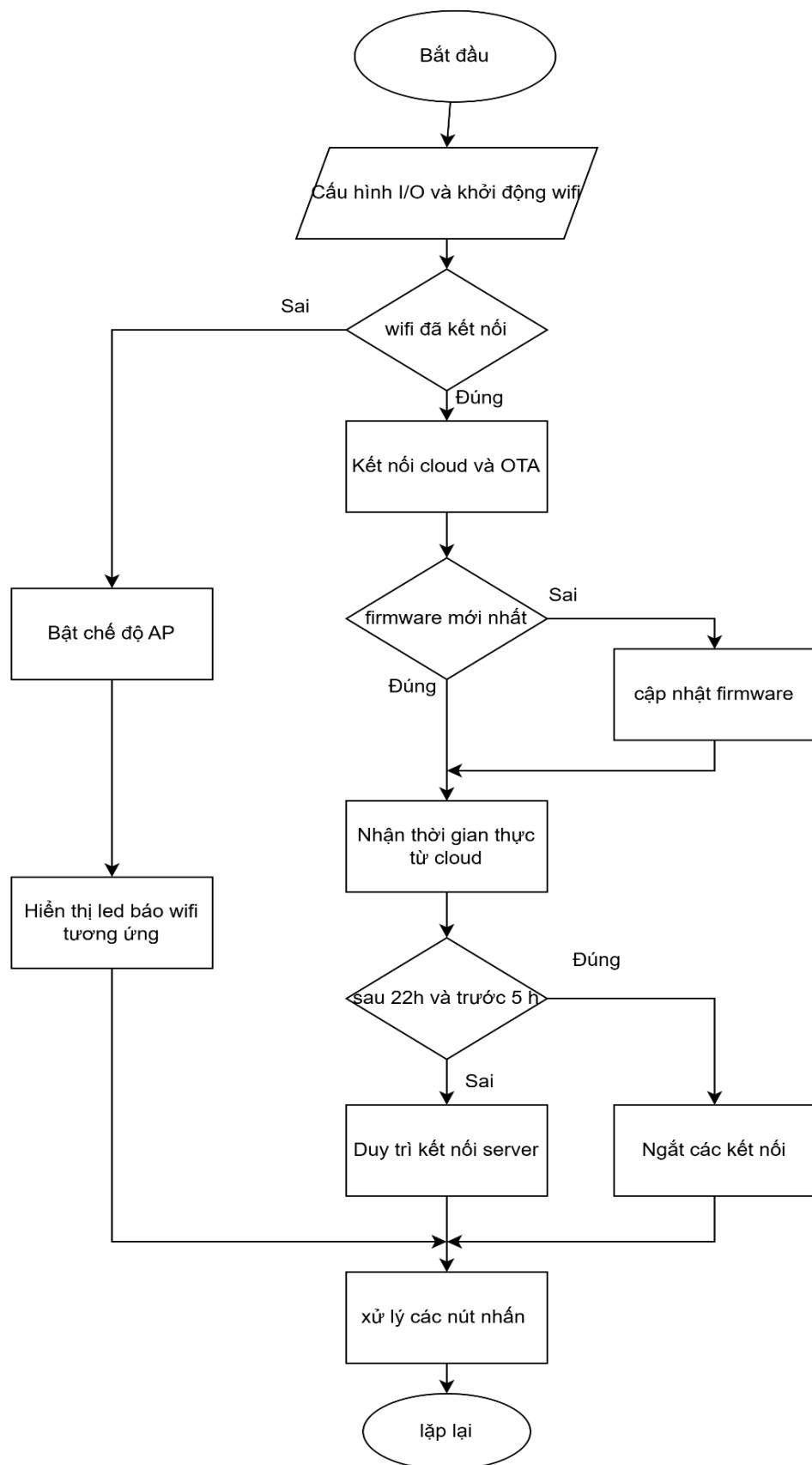
Hình ảnh 3d của mạch trên phần mềm:



Hình 3.3: Mạch 3D trên phần mềm

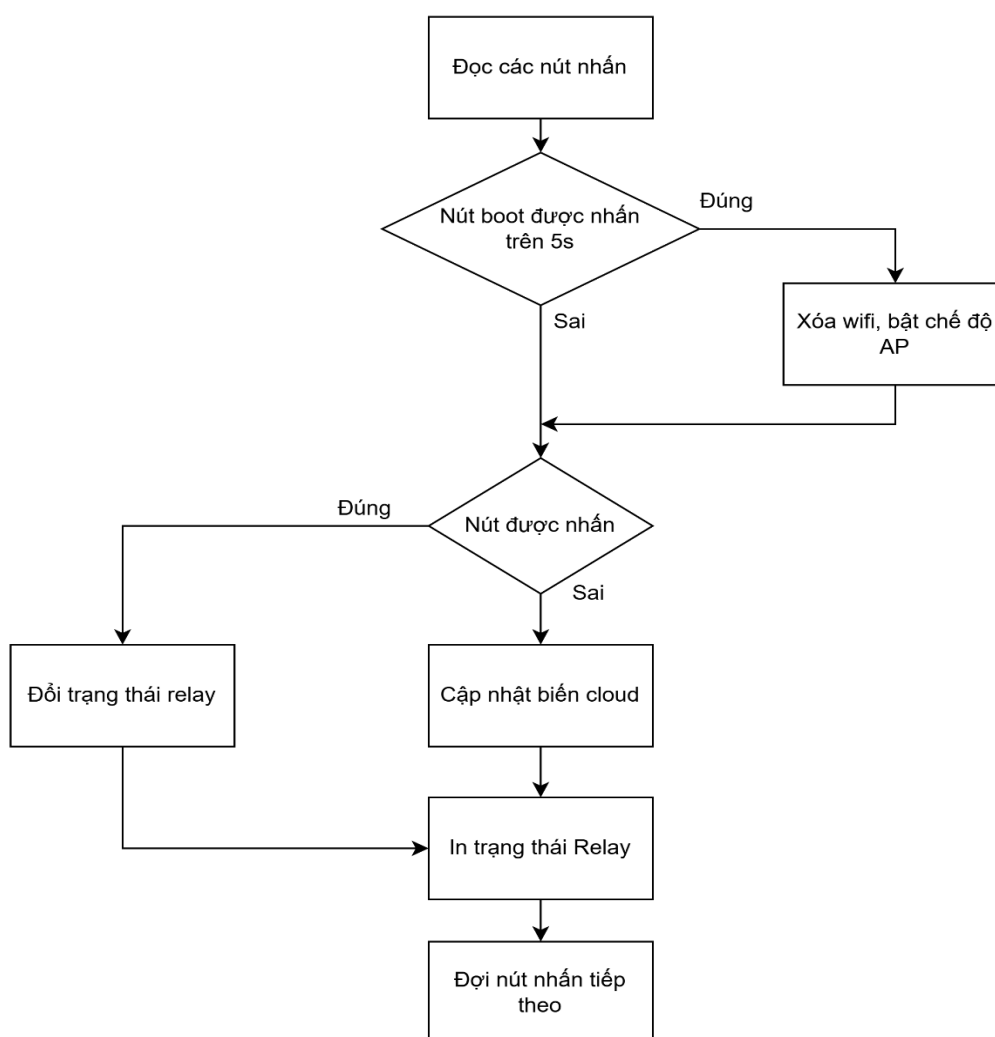
3.3. Thiết kế phần mềm

Hình dưới đây mô tả lưu đồ thuật toán của hệ thống:



Hình 3.4: Lưu đồ giải thuật tổng quát của hệ thống

Chương trình xử lý các nút nhấn được mô tả chi tiết bằng thuật toán sau:



Hình 3.5: Chương trình xử lý các nút nhấn

Mô tả thuật toán:

Hệ thống khởi động bằng việc cấu hình các chân I/O và kết nối WiFi. Nếu chưa kết nối được WiFi, thiết bị chuyển sang chế độ Access Point (AP) để cấu hình và hiển thị trạng thái bằng LED. Khi WiFi đã kết nối thành công, thiết bị kết nối lên cloud, kiểm tra và thực hiện cập nhật firmware OTA nếu có phiên bản mới.

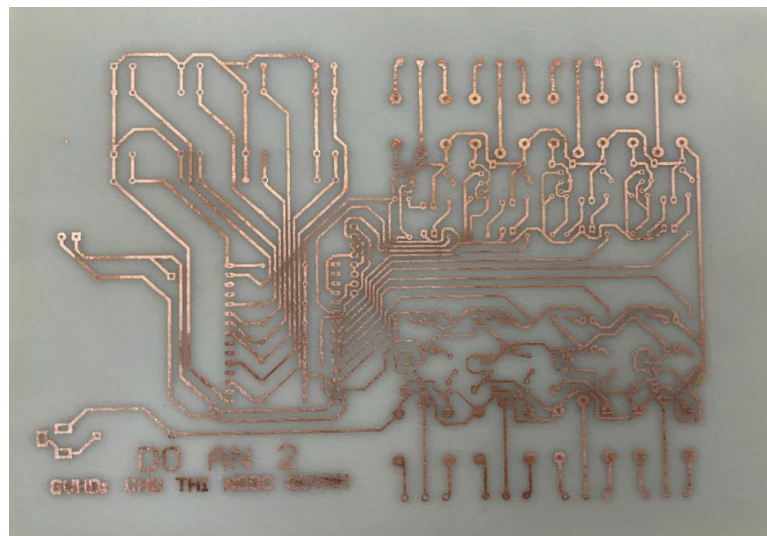
Sau đó, hệ thống đồng bộ thời gian thực từ cloud để xác định khung giờ hoạt động. Trong khoảng từ 22h đến trước 5h, thiết bị sẽ ngắt các kết nối nhằm tiết kiệm năng lượng; ngoài khoảng thời gian này, kết nối server được duy trì để xử lý các trạng thái hệ thống, sau đó lặp lại chu trình.

Song song, hệ thống liên tục đọc trạng thái các nút nhấn. Nếu nút boot được giữ trên 5 giây, thiết bị xóa cấu hình WiFi và chuyển sang chế độ AP. Với các nút điều khiển thông thường, khi nút được nhấn, trạng thái relay sẽ được thay đổi, cập nhật lên cloud và hiển thị trạng thái hiện tại, rồi chờ lần nhấn tiếp theo

3.4. Thi Công

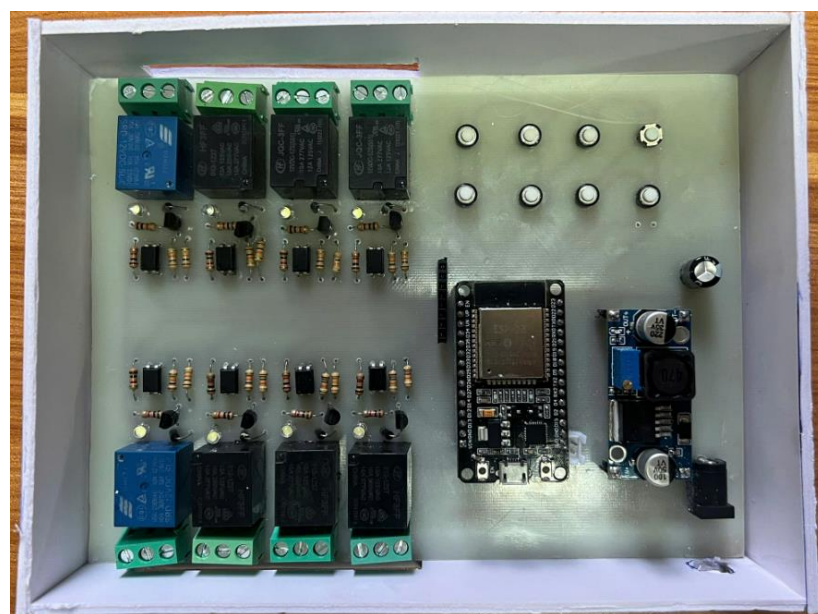
3.4.1. Thi công phần cứng

Khi chưa hàn linh kiện :



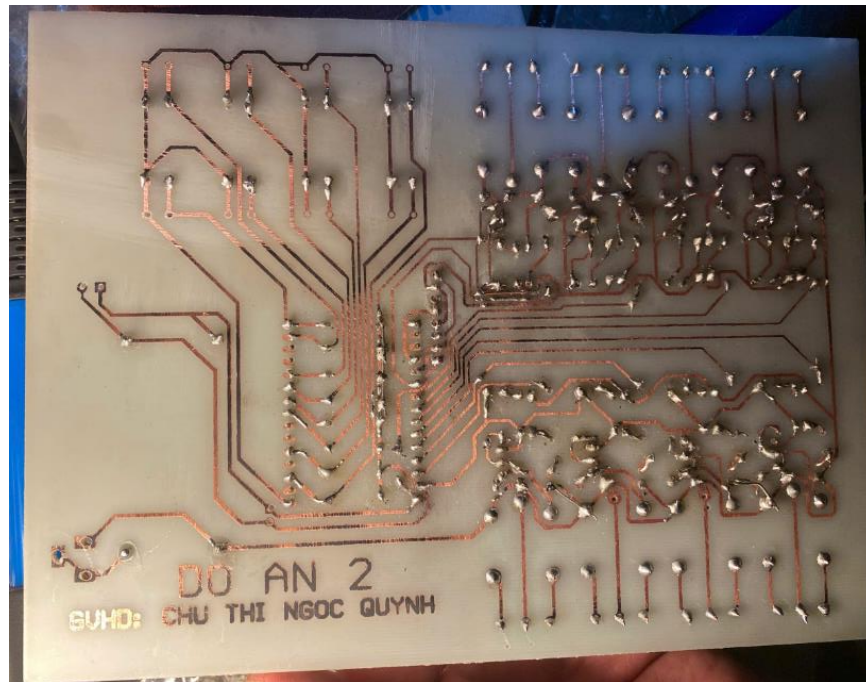
Hình 3.6: Mạch khi chưa hàn linh kiện

Khi hàn linh kiện, mặt trước:



Hình 3.7: Mặt trước

Khi hàn linh kiện, mặt sau:



Hình 3.8: Mặt sau

3.4.2. Thi công phần mềm

Phần mềm của hệ thống được xây dựng nhằm thực hiện chức năng giám sát và điều khiển thiết bị từ xa thông qua nền tảng Internet of Things (IoT). Quá trình thi công phần mềm bao gồm việc thiết lập môi trường phát triển, cấu hình hệ thống trên nền tảng Arduino IoT Cloud, tổ chức cấu trúc chương trình và quản lý mã nguồn, cũng như nạp và kiểm tra hoạt động của phần mềm trên vi điều khiển ESP32.

Do mã nguồn chương trình có dung lượng lớn và được quản lý tập trung trên nền tảng GitHub, mục này không trình bày chi tiết từng đoạn mã mà tập trung mô tả quy trình triển khai, cấu trúc chương trình và cách thức vận hành tổng thể của hệ thống phần mềm.

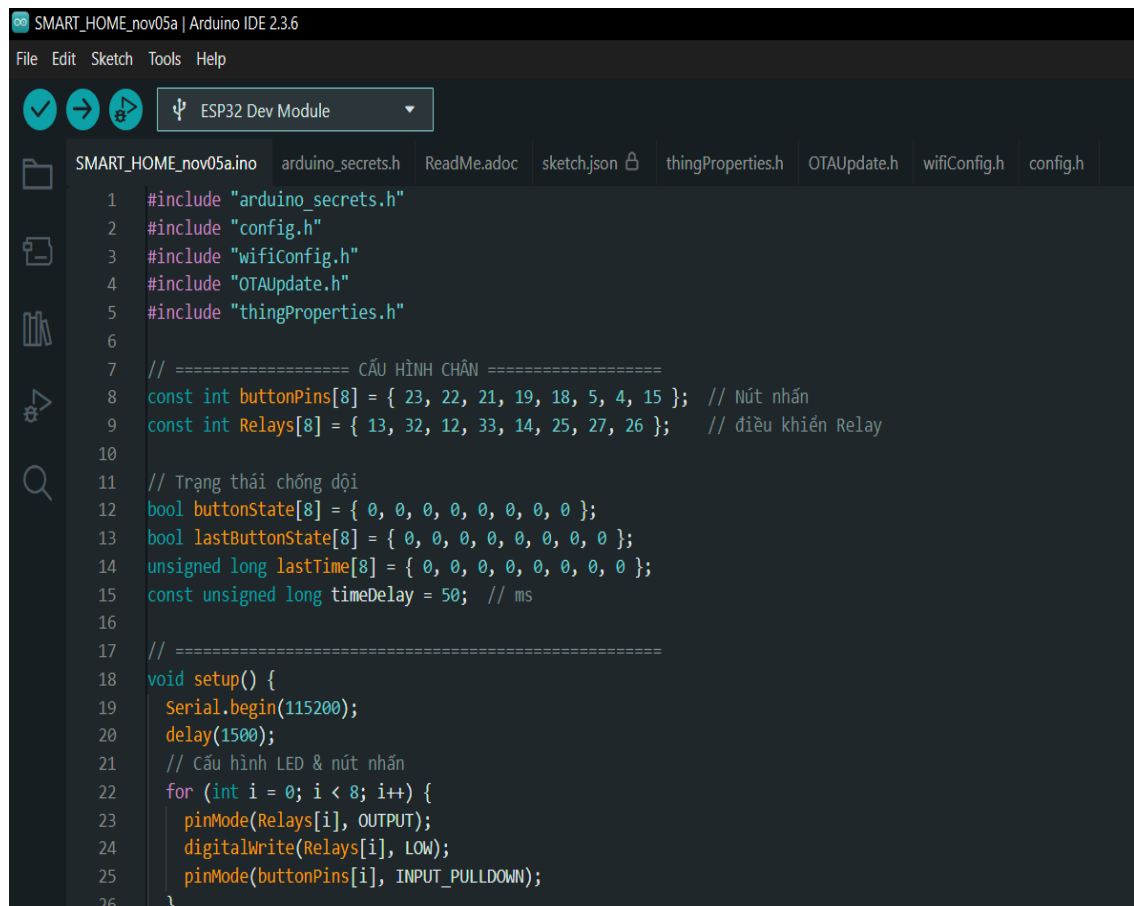
3.4.2.1. Thiết lập môi trường và thư viện

Trong quá trình thi công phần mềm, chương trình sử dụng các thư viện tiêu chuẩn của Arduino và ESP32 kết hợp với các module tự xây dựng nhằm đảm bảo khả năng kết nối mạng, giao tiếp với Arduino IoT Cloud và cập nhật firmware từ xa. Danh sách các thư viện và chức năng được trình bày trong Bảng 3.1 :

Bảng 3.1: Danh sách thư viện và chức năng sử dụng trong hệ thống

Thư viện	Chức năng
ArduinoIoTCloud.h	Kết nối ESP32 với nền tảng Arduino IoT Cloud và đồng bộ dữ liệu.
Arduino_ConnectionHandler.h	Quản lý và duy trì trạng thái kết nối mạng của hệ thống
thingProperties.h	Cấu hình Thing và các biến điều khiển trên Arduino IoT Cloud.
WiFi.h	Thiết lập và quản lý kết nối WiFi cho ESP32.
WiFiClientSecure.h	Hỗ trợ kết nối HTTPS bảo mật khi tải firmware từ GitHub.
HTTPClient.h	Thực hiện yêu cầu HTTP/HTTPS để tải firmware OTA.
OTAUpdate.h	Quản lý quy trình cập nhật firmware từ xa qua GitHub
wifiConfig.h	Quản lý cấu hình WiFi động cho thiết bị.
config.h	Lưu các tham số cấu hình chung hệ thống
Arduino_ConnectionHandler.h	Xử lý quá trình kết nối mạng (WiFi/Ethernet), tự động kết nối lại khi mất mạng
ArduinoIoTCloud.h	Quản lý giao tiếp giữa ESP32 và Arduino IoT Cloud
EEPROM.h	Cho phép lưu trữ và đọc dữ liệu cấu hình thông tin

Quá trình phát triển phần mềm được thực hiện trong môi trường Arduino với công cụ chính là Arduino IDE phiên bản 2.3. Đây là phiên bản hỗ trợ tốt cho dòng vi điều khiển ESP32, cho phép biên dịch và nạp chương trình một cách trực tiếp thông qua cổng USB–UART. Để lập trình cho ESP32, gói hỗ trợ ESP32 do Espressif Systems cung cấp được cài đặt thông qua Board Manager của Arduino IDE, giúp hệ thống nhận dạng đúng phần cứng và hỗ trợ đầy đủ các chức năng phần mềm cần thiết.



Hình 3.9: Giao diện Arduino IDE 2.3 và cấu hình Board ESP32

3.4.2.2. Cấu hình hệ thống trên Arduino IoT Cloud

Arduino IoT Cloud đóng vai trò trung tâm trong hệ thống giám sát và điều khiển từ xa. Trên nền tảng này, một thiết bị (Thing) được tạo để đại diện cho vi điều khiển ESP32 trong hệ thống. Thing này chứa toàn bộ thông tin cấu hình, trạng thái kết nối và các biến dữ liệu dùng để trao đổi thông tin giữa phần cứng và hệ thống Cloud.

Sau khi tạo Thing, các biến dữ liệu được khai báo nhằm đồng bộ trạng thái giữa thiết bị và Cloud. Trong hệ thống này, các biến điều khiển được sử dụng để quản lý trạng thái của các relay đầu ra. Mỗi relay tương ứng với một biến kiểu Boolean, cho phép người dùng thực hiện thao tác bật hoặc tắt thiết bị từ xa. Các biến này được Arduino IoT Cloud tự động sinh mã và ánh xạ vào chương trình nhúng thông qua tệp cấu hình tương ứng, do đó không cần khai báo thủ công trong mã nguồn chính.

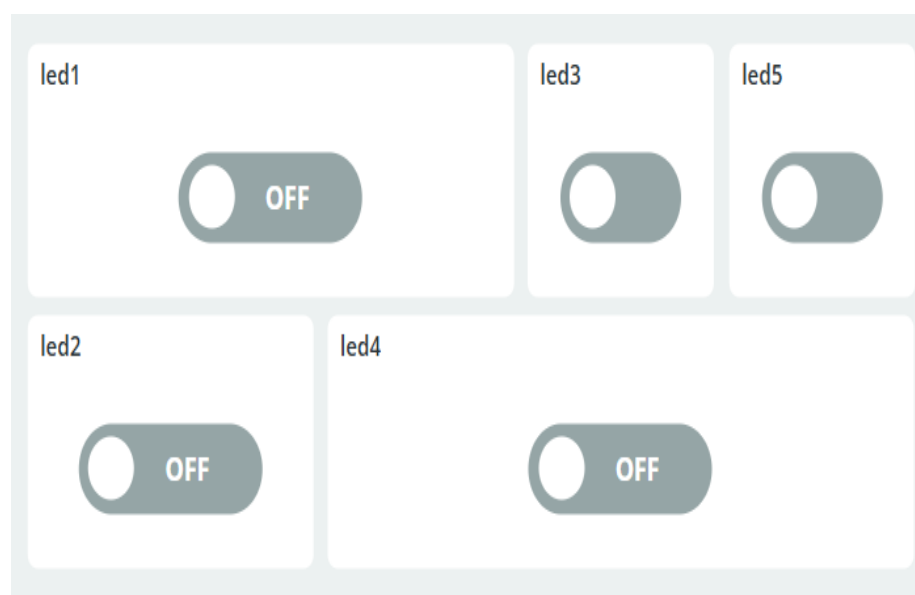
Cloud Variables

[ADD](#)

Name ↓	Last Value	Last Update	
☐ led1 CloudLight led1;	true	26 Dec 2025 13:37:12	
☐ led2 CloudLight led2;	false	26 Dec 2025 13:37:12	
☐ led3 CloudLight led3;	false	26 Dec 2025 13:37:12	
☐ led4 CloudLight led4;	true	26 Dec 2025 13:37:12	
☐ led5 CloudLight led5;	true	26 Dec 2025 13:37:12	

Hình 3.10: Thing và khai báo các biến trên Arduino IoT Cloud

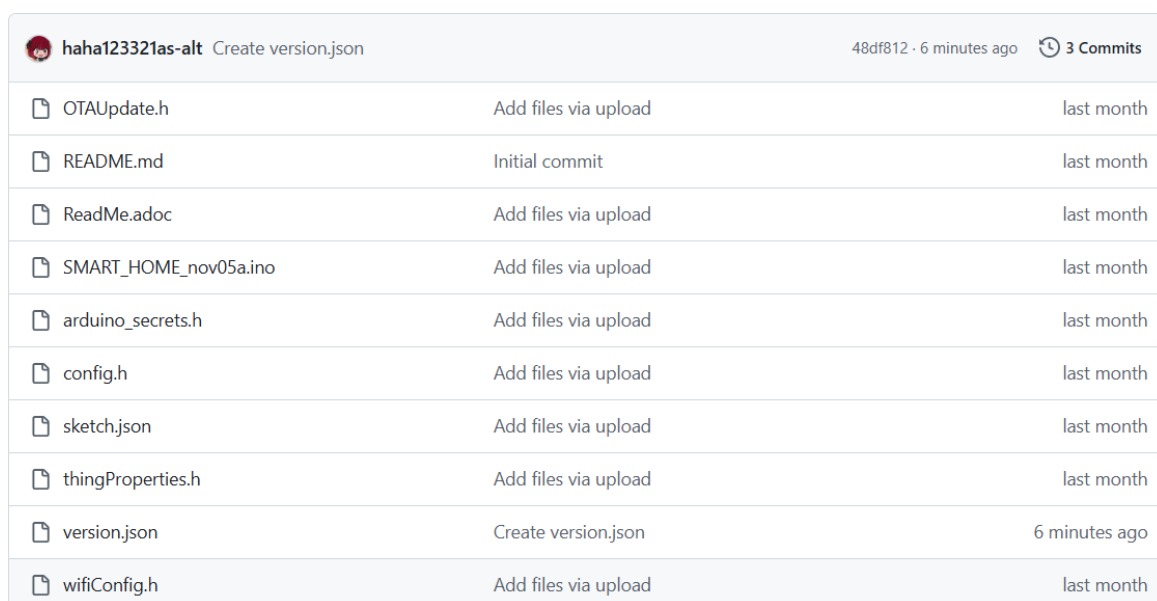
Trên cơ sở các biến đã khai báo, giao diện điều khiển Dashboard được thiết kế bằng các thành phần đồ họa có sẵn của Arduino IoT Cloud. Dashboard cho phép người dùng giám sát trạng thái hoạt động của hệ thống và điều khiển thiết bị thông qua trình duyệt web hoặc ứng dụng di động. Giao diện này đóng vai trò là cầu nối trực tiếp giữa người sử dụng và phần cứng, thể hiện rõ khả năng điều khiển và giám sát từ xa của hệ thống.



Hình 3.11: Dashboard điều khiển hệ thống trên Arduino IoT Cloud

3.4.2.3. Cấu trúc chương trình và quản lý mã nguồn trên GitHub

Trong quá trình phát triển phần mềm, toàn bộ mã nguồn của dự án được quản lý và lưu trữ tập trung trên nền tảng GitHub. Việc sử dụng GitHub cho phép theo dõi lịch sử chỉnh sửa, quản lý phiên bản và hỗ trợ quá trình cập nhật firmware từ xa thông qua cơ chế OTA. Thay vì trình bày mã nguồn chi tiết trong báo cáo, đường dẫn đến kho mã nguồn được cung cấp nhằm phục vụ công tác kiểm tra và đánh giá.



 haha123321as-alt	Create version.json	48df812 · 6 minutes ago	🕒 3 Commits
 OTAUpdate.h	Add files via upload		last month
 README.md	Initial commit		last month
 ReadMe.adoc	Add files via upload		last month
 SMART_HOME_nov05a.ino	Add files via upload		last month
 arduino_secrets.h	Add files via upload		last month
 config.h	Add files via upload		last month
 sketch.json	Add files via upload		last month
 thingProperties.h	Add files via upload		last month
 version.json	Create version.json		6 minutes ago
 wifiConfig.h	Add files via upload		last month

Hình 3.12: Kho mã nguồn của dự án được quản lý trên nền tảng GitHub

Chương trình được tổ chức theo cấu trúc mô-đun nhằm tăng tính rõ ràng và dễ bảo trì. Tập chương trình chính đảm nhiệm vai trò khởi tạo hệ thống và điều phối hoạt động tổng thể. Các tệp cấu hình riêng biệt được sử dụng để quản lý kết nối WiFi động, xử lý cập nhật firmware từ xa và lưu trữ thông tin kết nối với Arduino IoT Cloud cũng như các biến được đồng bộ dữ liệu. Cách tổ chức này giúp chương trình có cấu trúc rõ ràng, thuận tiện cho việc mở rộng và nâng cấp trong tương lai.

Đường dẫn đến mã nguồn của dự án được cung cấp như sau:

https://github.com/haha123321as-alt/ESP_32_Controller_8_relay

Mã nguồn được thiết lập ở chế độ công khai nhằm phục vụ quá trình kiểm tra và đánh giá của giảng viên.

3.4.2.4. Quy trình nạp và kiểm tra phần mềm

Trong quá trình kiểm tra và vận hành thử nghiệm, công cụ Serial Monitor được sử dụng để theo dõi trạng thái hoạt động của hệ thống. Các thông tin hiển thị bao gồm quá trình khởi tạo, trạng thái kết nối WiFi, trạng thái kết nối với nền tảng Cloud và hoạt động của các tiến trình chạy ngầm. Việc theo dõi các thông tin này giúp phát hiện sớm các lỗi phát sinh và đánh giá mức độ ổn định của hệ thống phần mềm.

```
07:06:25.930 -> WiFi name: chandoi
07:06:25.930 -> Password: bekhoai123456789
07:06:25.930 -> Kết nối tới WiFi đã lưu...
07:06:26.024 -> .....✅ Connected to WiFi!
07:06:28.264 -> 📶 IP Address: 192.168.0.108
07:06:28.528 ->
07:06:28.528 -> ✅ Kết nối thành công!
07:06:28.528 -> IP Address: 192.168.0.108
07:06:28.528 -> 🌐 Web server started!
07:06:28.528 -> ✅ WiFi OK, Start Cloud & OTA...
07:06:28.528 -> ***** Arduino IoT Cloud - 2.8.0 *****
07:06:28.528 -> Device ID: 9847066e-446f-4829-9124-e3c7f0b05840
07:06:28.528 -> MQTT Broker: iot.arduino.cc:8884
07:06:30.499 -> 🔍 Đang kiểm tra firmware mới...
07:06:31.378 -> 📌 Phiên bản hiện tại: 1.0.2
07:06:31.378 -> 📌 Phiên bản mới nhất: 1.0.2
07:06:31.378 -> ✅ Firmware đang ở phiên bản mới nhất.
07:06:31.378 -> WiFi.status(): 3
07:06:31.378 -> Connected to ""
07:06:31.410 -> Relay 6 bật
07:06:36.063 -> Connected to Arduino IoT Cloud
07:06:36.063 -> Thing ID: fee9dd01-bf0b-48e2-803a-5196486605c0
07:06:36.392 -> Relay 1 bật
07:06:36.392 -> Relay 2 bật
07:06:36.392 -> Relay 3 bật
07:06:36.392 -> Relay 4 bật
```

Hình 3.13: Theo dõi trạng thái hoạt động thông qua Serial Monitor

Trong giai đoạn triển khai ban đầu, chương trình được nạp cho vi điều khiển ESP32 thông qua cổng USB–UART bằng Arduino IDE. Sau khi nạp thành công, thiết bị tiến hành khởi tạo hệ thống, kết nối mạng WiFi và thiết lập kết nối với Arduino IoT Cloud. Từ thời điểm này, hệ thống có thể hoạt động độc lập và hỗ trợ cập nhật phần mềm từ xa thông qua OTA mà không cần kết nối trực tiếp với máy tính.

CHƯƠNG 4 : KẾT QUẢ VÀ ĐÁNH GIÁ

4.1. Mô hình thực tế

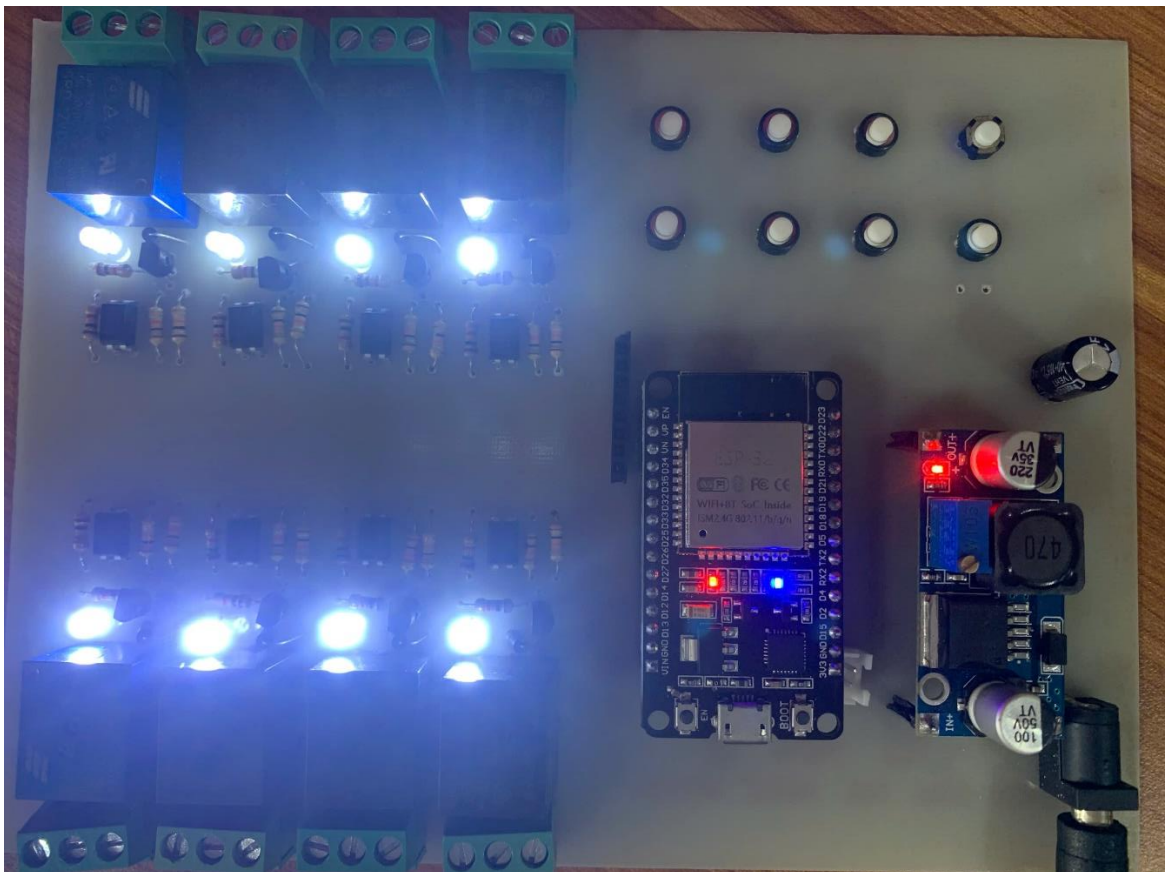
Hệ thống hoàn thiện bao gồm mạch điều khiển trung tâm kết nối với các bóng đèn dùng để mô phỏng tải tiêu thụ. Nguồn cấp cho mạch điều khiển sử dụng adapter 12 V–2 A. Các nguồn điện áp 220 V, 12 V và 5 V được sử dụng làm nguồn tải thử, nhằm kiểm tra và đánh giá hoạt động của hệ thống.

4.2. Kết quả đo kiểm

Tiến hành thử nghiệm hệ thống với các kịch bản:

Điều khiển tại chỗ: Nhấn tất cả nút bấm trên mạch, Relay đóng/ngắt tức thời, đèn sáng/tắt tương ứng, LED chỉ thị trên mạch hoạt động đúng.

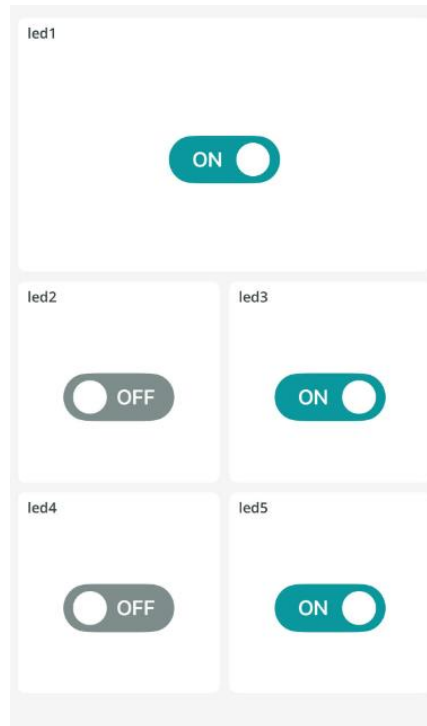
Hình dưới đây là kết quả của phép thử trên:



Hình 4.1: Kết quả phép thử điều khiển trực tiếp bằng nút nhấn

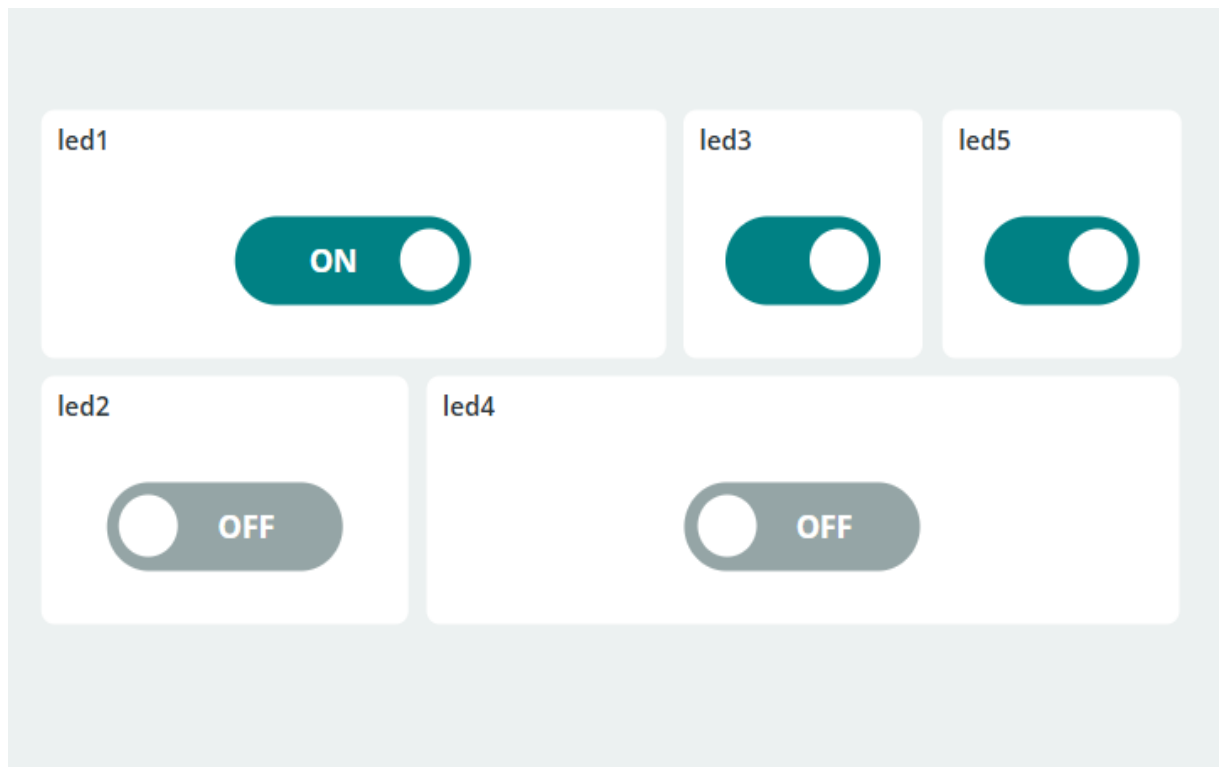
Điều khiển từ xa: Sử dụng điện thoại kết nối cùng mạng WiFi, truy cập giao diện điều khiển. Thao tác bật/tắt trên điện thoại có độ trễ rất thấp (< 0.5 giây).

Màn hình điều khiển thiết bị trên điện thoại:



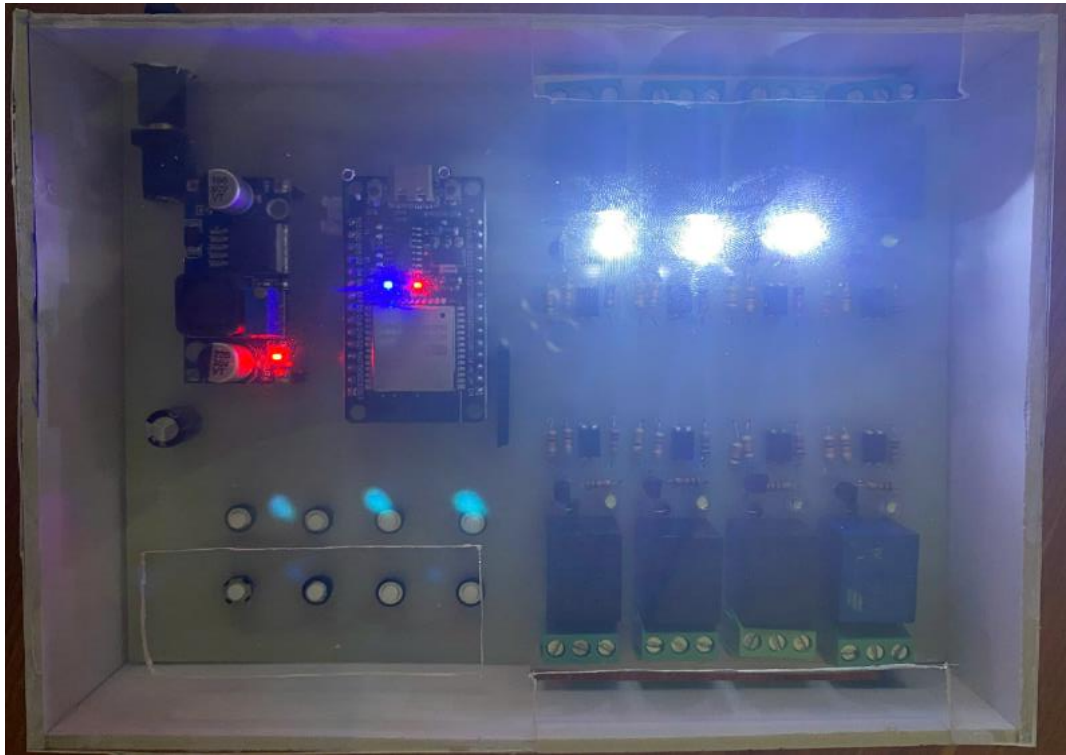
Hình 4.2: Giao diện điều khiển trên điện thoại

Màn hình điều khiển trên máy tính tương ứng



Hình 4.3: Màn hình điều khiển trên máy tính

Kết quả thực tế trên mạch:



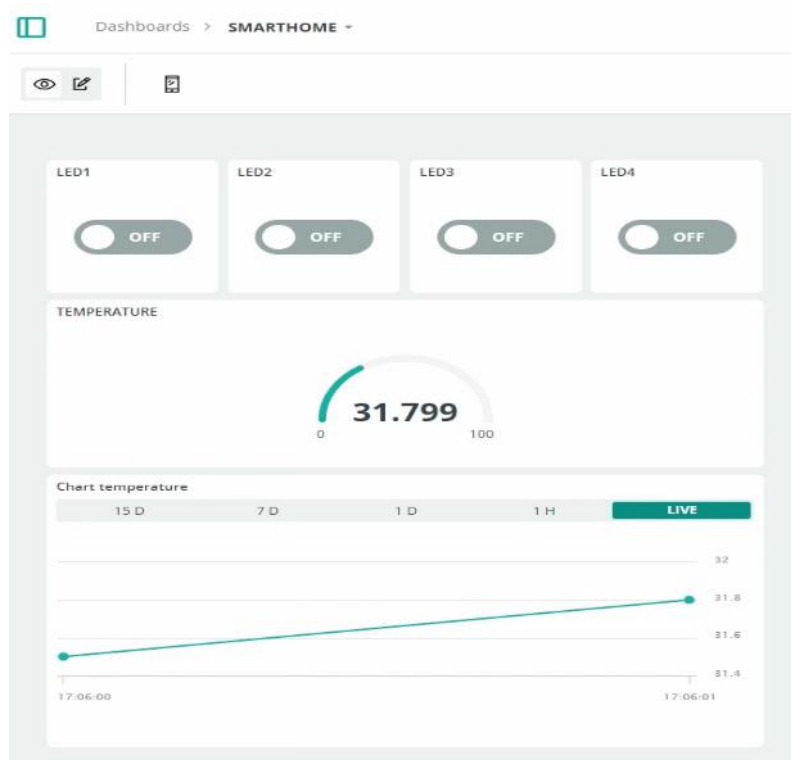
Hình 4.4: Kết quả của phép thử điều khiển qua WiFi

Thiết lập WiFi: Thực hiện cài đặt lại WiFi cho hệ thống bằng cách giữ liên tục nút nhấn boot trên ESP32, đèn báo trên esp32 sáng nhấp nháy và cho phép người cài đặt lại WiFi, khi người dùng cài đặt xong, đèn báo sáng liên tục báo hiệu WiFi đã kết nối thành công và có thể thực hiện điều khiển qua các ứng dụng Google Assistant,...

Hình 4.5: Kiểm tra cài đặt wifi và ngày giờ

Kiểm tra cài đặt giờ bật tắt: Thực hành cài đặt giờ bật tắt cho hệ thống, kết quả hệ thống tự động bật tắt chính xác trong sai số cho phép đã tính toán.

Kiểm tra OTA: Thực hiện kết nối mạch với cảm biến DHT11, chỉnh sửa lại version và chương trình trên github, reset hệ thống và kiểm tra lại, ta thấy hệ thống đã cập nhật thêm hiển thị giá trị nhiệt độ và độ ẩm trên đọc được từ DHT11:



Hình 4.6: Kết quả phép thử cập nhật hệ thống qua OTA

Độ ổn định: Cho hệ thống chạy liên tục trong 24 giờ, vi điều khiển không bị treo, IC nguồn LM2596 chỉ ấm nhẹ, không quá nhiệt.

4.3. Đánh giá

Về độ chính xác, mạch hoạt động ổn định và đáp ứng đúng các lệnh điều khiển từ hệ thống trung tâm, không ghi nhận hiện tượng tự kích hoạt hay sai lệch trạng thái do nhiễu điện từ trong quá trình thử nghiệm. Các kênh điều khiển hoạt động độc lập, đảm bảo tính nhất quán giữa lệnh điều khiển và trạng thái tải.

Về tính an toàn, hệ thống sử dụng opto cách ly PC817 giữa khối điều khiển và khối công suất. Khi tiến hành thử nghiệm tạo sự cố ngắn mạch nhẹ ở phía cuộn dây relay (giả lập), vi điều khiển ESP32 vẫn duy trì hoạt động bình thường, không

xảy ra hiện tượng treo hoặc reset. Kết quả này cho thấy mạch có khả năng cách ly tốt, giúp bảo vệ khối điều khiển trước các sự cố điện từ phía tải.

4.4. Các lỗi phát sinh và hướng khắc phục

Trong quá trình thi công và thử nghiệm, hệ thống gặp hiện tượng vi điều khiển ESP32 bị reset khi thực hiện đóng/ngắt đồng thời nhiều relay. Qua quá trình kiểm tra và đo đạc, nguyên nhân được xác định là do sụt áp tức thời tại đầu ra nguồn khi dòng khởi động của các relay tăng cao.

Để khắc phục hiện tượng trên, nhóm đã bổ sung tụ hóa dung lượng 470 μF tại đầu ra của khối nguồn hạ áp LM2596 nhằm tăng khả năng bù dòng tức thời. Sau khi áp dụng các biện pháp trên, hệ thống hoạt động ổn định, không còn xuất hiện hiện tượng reset khi đóng/ngắt nhiều relay cùng lúc.

KẾT LUẬN

Sau quá trình nghiên cứu, thiết kế và thi công, đồ án “Nghiên cứu xây dựng hệ thống điều khiển và giám sát thiết bị điện trong nhà sử dụng vi điều khiển ESP32” đã hoàn thành và đạt được các mục tiêu đề ra:

1. Ưu điểm

Thiết kế mạch đạt tiêu chuẩn công nghiệp, sử dụng nguồn xung hiệu suất cao kết hợp cách ly quang, đảm bảo an toàn và độ tin cậy trong quá trình vận hành. Hệ thống tận dụng sức mạnh của vi điều khiển ESP32, cho phép điều khiển từ xa qua kết nối WiFi một cách tiện lợi và ổn định.

Về mặt bảo mật, hệ thống hỗ trợ cơ chế bảo vệ trong quá trình cài đặt và cấu hình WiFi, hạn chế truy cập trái phép, đảm bảo an toàn cho người dùng và thiết bị.

Các tính năng nổi bật bao gồm điều khiển linh hoạt với 8 kênh độc lập, hỗ trợ nhiều chế độ làm việc khác nhau, tương thích với Google Assistant và Alexa. Hệ thống cho phép lắp đặt linh hoạt, tích hợp các chức năng thông minh như hẹn giờ, đếm ngược, chia sẻ thiết bị và tạo ngữ cảnh tự động.

Với cấu hình 8 kênh điều khiển, hệ thống đặc biệt phù hợp cho các không gian có diện tích vừa như phòng học, phòng làm việc hoặc căn hộ nhỏ, đáp ứng nhu cầu điều khiển đồng thời khoảng 8 thiết bị điện. Ngoài ra, chi phí linh kiện hợp lý, dễ thay thế và sửa chữa, thuận tiện cho việc bảo trì và mở rộng.

2. Nhược điểm

Mặc dù hệ thống đáp ứng tốt các yêu cầu điều khiển và tính linh hoạt, tuy nhiên vẫn còn một số hạn chế nhất định. Thứ nhất, hệ thống phụ thuộc vào kết nối WiFi, do đó hiệu quả điều khiển từ xa có thể bị ảnh hưởng khi mạng không ổn định hoặc bị gián đoạn. Thứ hai, số lượng kênh điều khiển cố định ở mức 8 kênh, chưa phù hợp cho các không gian lớn hoặc nhu cầu điều khiển số lượng thiết bị nhiều hơn.

Ngoài ra, các tính năng bảo mật hiện tại chủ yếu tập trung vào mức cơ bản trong quá trình cài đặt WiFi, chưa triển khai các cơ chế bảo mật nâng cao như mã hóa hai lớp hoặc xác thực đa yếu tố. Hệ thống cũng chưa tích hợp các cảm biến phản hồi trạng thái tải thực tế, nên việc giám sát mới dừng ở mức điều khiển đóng/cắt.

Cuối cùng, do sử dụng linh kiện phổ thông nhằm tối ưu chi phí, khả năng chịu tải và độ bền dài hạn của hệ thống vẫn cần được đánh giá thêm trong điều kiện vận hành thực tế.

3. Phương hướng phát triển

Trong thời gian tới, hệ thống sẽ tiếp tục được nghiên cứu và hoàn thiện nhằm nâng cao tính ổn định, an toàn và khả năng mở rộng. Trước hết, việc thay thế relay cơ bằng Triac hoặc SSR (Solid State Relay) sẽ được triển khai nhằm giảm kích thước mạch, tăng tuổi thọ linh kiện và loại bỏ hoàn toàn tiếng ồn trong quá trình đóng cắt tải.

Bên cạnh đó, hệ thống sẽ được tối ưu hóa về mặt năng lượng bằng cách loại bỏ các LED báo trạng thái không cần thiết, đồng thời áp dụng các chế độ tiết kiệm năng lượng cho vi điều khiển, giúp giảm điện năng tiêu thụ khi hệ thống hoạt động liên tục.

Về mặt phần mềm, một ứng dụng di động riêng sẽ được phát triển để giám sát và điều khiển hệ thống, cho phép theo dõi điện năng tiêu thụ theo thời gian thực, thống kê lịch sử sử dụng và cảnh báo khi vượt ngưỡng tiêu thụ cho phép.

Ngoài ra, hệ thống sẽ được nâng cấp các cơ chế bảo mật, bao gồm mã hóa dữ liệu truyền qua WiFi, xác thực người dùng và phân quyền truy cập, nhằm đảm bảo an toàn thông tin trong quá trình vận hành.

Cuối cùng, khả năng mở rộng số kênh điều khiển và tích hợp thêm các cảm biến đo dòng, điện áp hoặc công suất sẽ được nghiên cứu, giúp hệ thống không chỉ dừng lại ở chức năng đóng/cắt mà còn trở thành một giải pháp quản lý và giám sát điện năng thông minh, phù hợp cho các ứng dụng thực tế quy mô lớn hơn.

TÀI LIỆU KHAM KHẢO

- [1] IEEE, “IEEE Internet of Things Journal,” Institute of Electrical and Electronics Engineers (IEEE), Dec 2024. [Trực tuyến]. Available: <https://iee-iotj.org/>. [Đã truy cập 12/2025].
- [2] IEEE, “IEEE Access,” Institute of Electrical and Electronics Engineers
- [3] Espressif Systems, “ESP32 Technical Documentation,” Espressif Systems
- [4] Arduino, “Arduino Documentation,” Arduino SA, 2024. [Trực tuyến]. Available: <https://docs.arduino.cc/>. [Đã truy cập 12/2025].
- [5] Nguyễn Đăng Triều, “WiFi trên ESP32 (Arduino IDE),” 8/2015. [Trực tuyến]. Available: <https://viblo.asia/p/wifi-tren-esp32-arduino-ide-3RL1BKWpVao>. [Đã truy cập 12/2025].
- [6] zornlink, “esp32 串口, 蓝牙等其他方式OTA,” Available: <https://www.cnblogs.com/zornlink/p/11412232.html>. [Đã truy cập 12/2025].
- [7] PC817 Series Photocoupler Datasheet
- [8] ON Semiconductor, “2N2222A Bipolar Transistor Datasheet,” <https://www.onsemi.com/pdf/datasheet/2n2222a-d.pdf>, 2020.
- [9] OSEMI, “Axial-Lead Glass,” Semiconductor Components Industries, 2019. [Trực tuyến]. Available: <https://www.onsemi.com/>. [Đã truy cập 12/2025].

PHỤ LỤC

File Main.c

```
#include "arduino_secrets.h"

#include "config.h"

#include "OTAUpdate.h"

#include "wifiConfig.h"

#include "thingProperties.h"

const int buttonPins[8] = { 23, 22, 21, 19, 18, 5, 4, 15 };

const int Relays[8]      = { 13, 32, 12, 33, 14, 25, 27, 26 };

bool buttonState[8]      = { 0 };

bool lastButtonState[8] = { 0 };

unsigned long lastTime[8] = { 0 };

const unsigned long timeDelay = 50;

void setup() {

    Serial.begin(115200);

    delay(1500);

    Serial.println("🔌 ESP32 booted");

    for (int i = 0; i < 8; i++) {

        pinMode(Relays[i], OUTPUT);

        digitalWrite(Relays[i], LOW);

        pinMode(buttonPins[i], INPUT_PULLDOWN);

    }

    wifiConfigSetup();

    if (WiFi.status() == WL_CONNECTED) {

        Serial.println("✅ WiFi OK, Start Cloud & OTA...");

    }

}
```

```

    initProperties();

    ArduinoCloud.begin(ArduinoIoTPreferredConnection);

    setDebugMessageLevel(2);

    ArduinoCloud.printDebugInfo();

    initialOtaCheck();
} else {

    Serial.println("⚠ Mode AP. Skip Cloud & OTA.");

    Serial.println("Please connect ESP32-XXXX and access
http://192.168.4.1");

}

}

void loop() {

    wifiLoop();

    if (WiFi.status() == WL_CONNECTED) {

        ArduinoCloud.update();

    }

    handleButtons();

}

void handleButtons() {

    for (int i = 0; i < 8; i++) {

        int reading = digitalRead(buttonPins[i]);

        if (reading != lastButtonState[i]) { lastTime[i] = millis();

}

        if ((millis() - lastTime[i]) > timeDelay) {

            if (reading != buttonState[i]) {

                buttonState[i] = reading;

                if (reading == HIGH) {

```

```

        bool newState = !digitalRead(Relays[i]);
        digitalWrite(Relays[i], newState);
        switch (i) {
            case 0: led1 = newState; break;
            case 1: led2 = newState; break;
            case 2: led3 = newState; break;
            case 3: led4 = newState; break;
            case 4: led5 = newState; break;
        }

        Serial.print("Relay "); Serial.print(i + 1);
        Serial.println(newState ? " bật" : " tắt");
    }
}

lastButtonState[i] = reading;
}
}

void onLed1Change() {
    digitalWrite(Relays[0], led1);
    Serial.println(led1 ? "Relay 1 bật" : "Relay 1 tắt");
}

void onLed2Change() {
    digitalWrite(Relays[1], led2);
    Serial.println(led2 ? "Relay 2 bật" : "Relay 2 tắt");
}

void onLed3Change() {

```

```

    digitalWrite(Relays[2], led3);

    Serial.println(led3 ? "Relay 3 bật" : "Relay 3 tắt");
}

void onLed4Change() {
    digitalWrite(Relays[3], led4);

    Serial.println(led4 ? "Relay 4 bật" : "Relay 4 tắt");
}

void onLed5Change() {
    digitalWrite(Relays[4], led5);

    Serial.println(led5 ? "Relay 5 bật" : "Relay 5 tắt");
}

```

FILE wifiConfig.h

```

#ifndef WIFI_CONFIG_H
#define WIFI_CONFIG_H

#include <WiFi.h>
#include <EEPROM.h>
#include <WebServer.h>
#include <ArduinoJson.h>
#include <time.h>
#include "config.h"

// Cần include ArduinoIoTCloud để lấy giờ chính xác
#include <ArduinoIoTCloud.h>

#ifndef LED_PIN
#define LED_PIN 2
#endif

#ifndef BUTTON_PIN
#define BUTTON_PIN 0

```

```

#endif

WebServer server(80);

// Biến toàn cục

String ssid;

String password;

String onTimeStr = "00:00";

String offTimeStr = "00:00";

int wifiMode = 0; // 0: AP, 1: Connected

bool wifiCleared = false;

unsigned long buttonPressStart = 0;

bool buttonHeld = false;

bool isAuthenticated = false;

unsigned long wifiConnectedTime = 0;

bool hasTriggeredSleep = false;

const char LOGIN_PAGE[] PROGMEM = R"rawliteral(

<!DOCTYPE html>

<html lang="vi">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-
scale=1">

    <title>Đăng Nhập KMA</title>

    <style>

        :root { --primary-red: #A81F23; --accent-gold: #D4AF37; --bg-
light: #F4F7F6; }

        body { font-family: 'Segoe UI', sans-serif; background: var(-
-bg-light); display: flex; justify-content: center; align-items:
center; height: 100vh; margin: 0; }

```

```
.card { background: white; padding: 40px; border-radius: 20px;
box-shadow: 0 20px 40px rgba(0,0,0,0.1); text-align: center; width:
90%; max-width: 350px; border-top: 6px solid var(--primary-red); }
```

```
.logo { width: 80px; height: 80px; background: var(--primary-
red); border: 3px solid var(--accent-gold); border-radius: 50%;
display: flex; align-items: center; justify-content: center; margin:
0 auto 20px; color: var(--accent-gold); font-weight: bold; font-size:
16px; box-shadow: 0 4px 10px rgba(0,0,0,0.2); }
```

```
h2 { color: var(--primary-red); margin-bottom: 5px; font-size:
22px; text-transform: uppercase; font-weight: 700; }
```

```
input[type=password] { width: 100%; padding: 12px; margin-
bottom: 20px; border: 2px solid #eee; border-radius: 25px; outline:
none; text-align: center; transition: 0.3s; font-size: 16px; }
```

```
input[type=password]:focus { border-color: var(--primary-
red); background: #fff; }
```

```
input[type=submit] { width: 100%; background: var(--primary-
red); color: white; border: none; padding: 12px; border-radius: 25px;
font-weight: bold; cursor: pointer; text-transform: uppercase;
transition: 0.3s; box-shadow: 0 5px 15px rgba(168, 31, 35, 0.3); }
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="card">
```

```
<div class="logo">KMA</div>
```

```
<h2>Bảo Mật</h2>
```

```
<form action="/auth" method="post">
```

```
<input type="password" name="pass" placeholder="Nhập mật
khẩu..." required>
```

```
<input type="submit" value="Đăng Nhập">
```

```
</form>
```

```

        </div>

</body>

</html>

)rawliteral";

const char HTML_PAGE[] PROGMEM = R"rawliteral(

<!DOCTYPE html>

<html lang="vi">

<head>

    <meta charset="utf-8">

    <meta name="viewport" content="width=device-width, initial-
scale=1, maximum-scale=1, user-scalable=no">

    <title>KMA WIFI CONFIGURATION</title>

    <style>

        :root { --primary-red: #A81F23; --accent-gold: #D4AF37; -
-bg-light: #F4F7F6; --text-dark: #2D3436; --text-light: #FFFFFF; --
input-bg: #FFFFFF; }

        * { box-sizing: border-box; margin: 0; padding: 0; }

        body { font-family: 'Segoe UI', Roboto, Helvetica, Arial,
sans-serif; background-color: var(--bg-light); display: flex;
justify-content: center; align-items: center; min-height: 100vh;
padding: 20px; color: var(--text-dark); }

        .main-container { display: flex; background-color: #fff;
border-radius: 20px; box-shadow: 0 20px 40px rgba(0,0,0,0.1);
overflow: hidden; width: 100%; max-width: 900px; min-height: 550px; }

        .left-section { flex: 1; background: linear-
gradient(135deg, var(--primary-red) 0%, #80171a 100%); color: var(--
text-light); padding: 40px; display: flex; flex-direction: column;
justify-content: center; position: relative; }

        .school-logo-css { width: 80px; height: 80px; background-
color: var(--primary-red); border: 3px solid var(--accent-gold);

```

```

border-radius: 50%; display: flex; align-items: center; justify-
content: center; font-weight: bold; font-size: 14px; text-align:
center; margin-bottom: 20px; box-shadow: 0 4px 10px rgba(0,0,0,0.2);
}

.school-logo-css span { color: var(--accent-gold); }

.project-title { font-size: 32px; font-weight: 700;
margin-bottom: 10px; letter-spacing: 1px; }

.project-subtitle { font-size: 18px; margin-bottom: 40px;
opacity: 0.9; color: var(--accent-gold); }

.info-box { background: rgba(255, 255, 255, 0.1); padding:
20px; border-radius: 15px; border-left: 4px solid var(--accent-gold);
}

.info-box p { margin-bottom: 10px; font-size: 15px; line-
height: 1.5; }

.info-box strong { color: var(--accent-gold); font-weight:
600; }

.right-section { flex: 1; padding: 40px; background-color:
var(--input-bg); display: flex; flex-direction: column; justify-
content: center; }

.form-title { font-size: 24px; color: var(--primary-red);
margin-bottom: 5px; text-align: center; font-weight: 700; }

#info { text-align: center; font-size: 14px; color: #666;
margin-bottom: 30px; font-style: italic; }

.form-group { margin-bottom: 20px; }

label { display: block; font-size: 14px; font-weight: 600;
color: var(--text-dark); margin-bottom: 8px; }

select, input { width: 100%; height: 50px; padding: 0
15px; font-size: 16px; border: 2px solid #e0e0e0; border-radius: 10px;
background-color: #fcfcfc; transition: all 0.3s; outline: none; }

input[type="time"] { font-family: 'Segoe UI', sans-serif;
font-size: 18px; color: #333; }

```

```

        select:focus, input:focus { border-color: var(--primary-red); box-shadow: 0 0 0 3px rgba(168, 31, 35, 0.1); }

        .btn-group { display: flex; flex-direction: column; gap: 15px; margin-top: 20px; }

        button { width: 100%; height: 50px; border: none; border-radius: 25px; font-size: 16px; font-weight: 700; cursor: pointer; transition: all 0.3s ease; text-transform: uppercase; }

        .btn-primary { background-color: var(--primary-red); color: white; box-shadow: 0 4px 15px rgba(168, 31, 35, 0.3); }

        .btn-primary:hover { background-color: #8a191c; transform: translateY(-2px); }

        .btn-secondary { background-color: transparent; color: #666; border: 2px solid #e0e0e0; }

        .btn-secondary:hover { border-color: var(--text-dark); color: var(--text-dark); background-color: #f0f0f0; }

        .modal-overlay { position: fixed; top: 0; left: 0; width: 100%; height: 100%; background: rgba(0, 0, 0, 0.6); display: flex; justify-content: center; align-items: center; z-index: 2000; opacity: 0; visibility: hidden; transition: all 0.3s ease; backdrop-filter: blur(4px); }

        .modal-overlay.active { opacity: 1; visibility: visible; }

        .modal-box { background: white; padding: 30px; border-radius: 20px; width: 90%; max-width: 400px; text-align: center; transform: scale(0.8); transition: all 0.3s cubic-bezier(0.175, 0.885, 0.32, 1.275); box-shadow: 0 20px 60px rgba(0,0,0,0.3); border-top: 6px solid var(--primary-red); }

        .modal-overlay.active .modal-box { transform: scale(1); }

        .modal-icon { font-size: 40px; margin-bottom: 15px; }

        .modal-title { font-size: 22px; font-weight: 700; color: var(--text-dark); margin-bottom: 10px; }

```

```

        .modal-message { font-size: 16px; color: #555; margin-
bottom: 25px; line-height: 1.5; }

        .modal-actions { display: flex; gap: 10px; justify-
content: center; }

        .modal-btn { padding: 12px 25px; border-radius: 25px;
font-size: 14px; text-transform: uppercase; font-weight: 700; border:
none; cursor: pointer; flex: 1; transition: background 0.2s; }

        .btn-confirm { background: var(--primary-red); color:
white; }

        .btn-confirm:hover { background: #8a191c; }

        .btn-cancel { background: #e0e0e0; color: #555; }

        .btn-cancel:hover { background: #d0d0d0; }

        @media (max-width: 768px) { .main-container { flex-
direction: column; max-width: 400px; } .left-section { padding: 30px;
align-items: center; text-align: center; } .info-box { text-align:
left; width: 100%; } .right-section { padding: 30px; } }

    </style>

</head>

<body>

    <div class="modal-overlay" id="customModal">

        <div class="modal-box">

            <div class="modal-icon" id="m-icon">  </div>

            <div class="modal-title" id="m-title">Thông báo</div>

            <div class="modal-message" id="m-message">Nội
dung...</div>

            <div class="modal-actions">

                <button class="modal-btn btn-cancel" id="btn-
cancel" onclick="closeModal()">Hủy</button>

```

```

        <button class="modal-btn btn-confirm" id="btn-ok"
onclick="handleOk()">Đồng ý</button>

    </div>

</div>

<div class="main-container">

    <div class="left-section">

        <div class="school-logo-css"><span>KMA</span></div>

        <h1 class="project-title">ĐỒ ÁN 2</h1>

        <h2 class="project-subtitle">HỌC VIỆN KỸ THUẬT MẬT
Mã</h2>

        <div class="info-box">

            <p><strong>GVHD:</strong> Chu Thị Ngọc Quỳnh</p>

            <div style="margin-top: 15px;"></div>

            <p><strong>SVTH:</strong></p>

            <p>Hà Văn Đạt - DT060209</p>

            <p>Dương Hải Đăng - DT060206</p>

            <p>Nguyễn Mạnh Lân - DT060231</p>

        </div>

    </div>

    <div class="right-section">

        <h3 class="form-title">CÀI ĐẶT KẾT NỐI</h3>

        <p id="info">Đang quét mạng WiFi...</p>

        <div class="form-group">

            <label for="ssid">Chọn mạng WiFi:</label>

            <div style="position: relative;">

                <select id="ssid"><option>Vui lòng
đợi...</option></select>

```

```

        <span style="position: absolute; right: 15px;
top: 50%; transform: translateY(-50%); color: #888; pointer-events:
none;">▼</span>

    </div>

</div>

<div class="form-group">

    <label for="password">Mật khẩu:</label>

        <input id="password" type="password"
placeholder="Nhập mật khẩu WiFi">

    </div>

<div style="display: flex; gap: 15px;">

    <div class="form-group" style="flex: 1;">

        <label for="onTime">Giờ BẬT (ON):</label>

        <input id="onTime" type="time">

    </div>

    <div class="form-group" style="flex: 1;">

        <label for="offTime">Giờ TẮT (OFF):</label>

        <input id="offTime" type="time">

    </div>

</div>

<div class="btn-group">

    <button type="button" class="btn-primary"
onclick="saveWifi()">LƯU CẤU HÌNH</button>

    <button type="button" class="btn-secondary"
onclick="reStart()">KHỞI ĐỘNG LẠI</button>

</div>

</div>

</div>

```

```

<script>

    window.onload=function(){ scanWifi(); }

    let okCallback = null;

    function showModal(title, message, type, callback = null)
{
    document.getElementById('m-title').innerText = title;
document.getElementById('m-message').innerText = message;

    const icon = document.getElementById('m-icon'); const
btnCancel = document.getElementById('btn-cancel'); const btnOk =
document.getElementById('btn-ok');

    if (type === 'success') { icon.innerText = '✅';
btnCancel.style.display = 'none'; btnOk.innerText = "OK";
btnOk.style.width = "100%"; }

    else if (type === 'confirm') { icon.innerText = '❓';
btnCancel.style.display = 'block'; btnOk.innerText = "Đồng ý"; }

    else if (type === 'error') { icon.innerText = '⚠️';
btnCancel.style.display = 'none'; btnOk.innerText = "Thử lại"; }

        okCallback = callback;
document.getElementById('customModal').classList.add('active');

    }

    function closeModal() {
document.getElementById('customModal').classList.remove('active'); }

    function handleOk() { closeModal(); if (okCallback)
okCallback(); }

    function scanWifi(){

        var xhr = new XMLHttpRequest();

        xhr.onreadystatechange = function(){

            if(xhr.readyState==4&&xhr.status==200){

```

```

        var data = xhr.responseText;
document.getElementById("info").innerHTML = "Đã tìm thấy các mạng WiFi gần đây.";

        var obj = JSON.parse(data); var select = document.getElementById("ssid"); select.innerHTML = "";

        if(obj.length === 0){ select[select.length] = new Option("Không tìm thấy WiFi", ""); }

        else { select[select.length] = new Option("Chọn WiFi của bạn...", ""); for(var i=0; i<obj.length;++i){ select[select.length] = new Option(obj[i],obj[i]); } }

    }

}

xhr.open("GET","/scanWifi",true); xhr.send();

}

function saveWifi(){

    let ssidVal = document.getElementById("ssid").value;
let passVal = document.getElementById("password").value; let onTimeVal = document.getElementById("onTime").value; let offTimeVal = document.getElementById("offTime").value;

    if(ssidVal === "" || ssidVal === "Vui lòng đợi...") { showModal("Lỗi", "Vui lòng chọn tên mạng WiFi!", "error"); return; }

    if(onTimeVal === "") onTimeVal = "00:00"; if(offTimeVal === "") offTimeVal = "00:00";

    document.getElementById("info").innerHTML = "Đang gửi dữ liệu...";

    var xhrSave = new XMLHttpRequest();

    xhrSave.onreadystatechange = function(){

        if(xhrSave.readyState==4){

            if(xhrSave.status==200){

```

```

        showModal("Thành Công!", "Đã lưu lịch:\nBật
lúc: " + onTimeVal + "\nTắt lúc: " + offTimeVal + "\n\nHãy khởi động
lại.", "success");

        document.getElementById("info").innerHTML
= "Đã lưu thành công! Hãy khởi động lại.";

    }

}

}

        var url = "/saveWifi?ssid=" +
encodeURIComponent(ssidVal) + "&pass=" + encodeURIComponent(passVal)
+ "&on=" + encodeURIComponent(onTimeVal) + "&off=" +
encodeURIComponent(offTimeVal);

        xhrSave.open("GET", url, true); xhrSave.send();

    }

    function reStart(){

        showModal("Xác Nhận", "Bạn có chắc chắn muốn khởi động
lại ESP32 không?", "confirm", function() {

            var xhr = new XMLHttpRequest();

            xhr.open("GET", "/reStart", true); xhr.send();

        });

    }

</script>

</body>

</html>

)rawliteral";

    void handleRoot() { if (!isAuthenticated) server.send(200,
"text/html", LOGIN_PAGE); else server.send(200, "text/html",
HTML_PAGE); }

```

```

    void handleAuth() { String inputPass = server.arg("pass"); if
(inputPass == WEB_PASSWORD) { isAuthenticated = true; server.send(200,
"text/html", "<script>location.href='/</script>"); } else {
server.send(200, "text/html", "<script>alert('Mat khau sai !!!');
location.href='/</script>"); } }

    void handleScanWifi() { int n = WiFi.scanNetworks();
DynamicJsonDocument doc(2048); for(int i=0; i<n; ++i){
doc.add(WiFi.SSID(i)); } String wifiList; serializeJson(doc,
wifiList); server.send(200, "application/json", wifiList);
WiFi.scanDelete(); }

    void handleSaveWifi() {

        String newSsid = server.arg("ssid"); String newPass =
server.arg("pass"); String newOn = server.arg("on"); String newOff =
server.arg("off");

        EEPROM.writeString(0, newSsid); EEPROM.writeString(32,
newPass); EEPROM.writeString(100, newOn); EEPROM.writeString(120,
newOff); EEPROM.commit();

        server.send(200, "text/plain", "Đã lưu!");

    }

    void handleRestart() { server.send(200, "text/plain", "OK");
delay(1000); ESP.restart(); }

    void checkSchedule() {

        if (wifiMode != 1) return;

        unsigned long epoch = ArduinoCloud.getLocalTime();

        if(epoch < 1000000000) return;

        struct tm timeinfo;

        localtime_r((time_t *)&epoch, &timeinfo);

```

```

int currentH = timeinfo.tm_hour;
int currentM = timeinfo.tm_min;

int offH, offM; sscanf(offTimeStr.c_str(), "%d:%d", &offH,
&offM);

int onH, onM; sscanf(onTimeStr.c_str(), "%d:%d", &onH, &onM);
if(offH == onH && offM == onM) return;

static int lastDebugMin = -1;

if (currentM != lastDebugMin) {

    Serial.printf("🕒 %02d:%02d | OFF: %02d:%02d | ON:
%02d:%02d\n", currentH, currentM, offH, offM, onH, onM);

    lastDebugMin = currentM;
}

if (currentH == offH && currentM == offM && !hasTriggeredSleep)
{

    Serial.println("🔴 GIỜ TẮT -> NGỦ...");

    int minutesNow = offH * 60 + offM;

    int minutesWake = onH * 60 + onM;

    int sleepMinutes = 0;

    if (minutesWake > minutesNow) { sleepMinutes = minutesWake
- minutesNow; }

    else { sleepMinutes = (1440 - minutesNow) + minutesWake;
}

    if (sleepMinutes <= 0) sleepMinutes = 1;

    Serial.printf("😴 Ngủ trong %d phút.\n", sleepMinutes);

    hasTriggeredSleep = true;

    delay(1000);

    uint64_t sleepMicroseconds = (uint64_t)sleepMinutes * 60
* 1000000ULL;

```

```

        esp_sleep_enable_timer_wakeup(sleepMicroseconds);

        esp_deep_sleep_start();

    }

    if (currentM != offM) { hasTriggeredSleep = false; }
}

void wifiConfigSetup() {

    pinMode(LED_PIN, OUTPUT); pinMode(BUTTON_PIN, INPUT_PULLUP);

    EEPROM.begin(512);

    ssid = EEPROM.readString(0); password = EEPROM.readString(32);

    onTimeStr = EEPROM.readString(100);    offTimeStr =
EEPROM.readString(120);

    ssid.trim();    password.trim();    onTimeStr.trim();
offTimeStr.trim();

    server.on("/", handleRoot); server.on("/auth", HTTP_POST,
handleAuth);

    server.on("/scanWifi", HTTP_GET, handleScanWifi);
server.on("/saveWifi", HTTP_GET, handleSaveWifi);
server.on("/reStart", HTTP_GET, handleRestart);

    if (ssid.length() > 0) {

        WiFi.mode(WIFI_STA);    WiFi.begin(ssid.c_str(),
password.c_str());

        unsigned long start = millis();

        while (WiFi.status() != WL_CONNECTED && millis() - start <
10000) { delay(500); digitalWrite(LED_PIN, !digitalRead(LED_PIN)); }

        if (WiFi.status() == WL_CONNECTED) { wifiMode = 1;
server.begin(); } else { WiFi.disconnect(); goto start_ap; }

    } else {

        start_ap: WiFi.mode(WIFI_AP_STA);

        uint8_t mac[6]; WiFi.softAPmacAddress(mac);

```

```

        String apName = "ESP32-" + String(mac[4], HEX) +
String(mac[5], HEX); apName.toUpperCase();

        WiFi.softAP(apName.c_str()); wifiMode = 0; server.begin();

    }

}

void wifiLoop() {
    server.handleClient();

    if (digitalRead(BUTTON_PIN) == LOW) {
        if (buttonPressStart == 0) buttonPressStart = millis();
        if (millis() - buttonPressStart > 5000 && !buttonHeld) {
            buttonHeld = true; for(int i=0; i<512; i++) EEPROM.write(i,
0); EEPROM.commit(); delay(500); ESP.restart();
        }
    } else { buttonPressStart = 0; buttonHeld = false; }
    // --- LOGIC LED MỚI: CHỚP TẮT 3 GIÂY KHI MẤT WIFI ---
    static unsigned long lastBlink = 0;
    if (wifiMode == 1) {
        // Đã kết nối -> Sáng liên tục
        digitalWrite(LED_PIN, HIGH);
    } else {
        // Chưa kết nối -> Chớp tắt mỗi 3 giây
        if (millis() - lastBlink > 3000) {
            lastBlink = millis();
            digitalWrite(LED_PIN, !digitalRead(LED_PIN));
        }
    }
}

```

```

        checkSchedule();
    }

#endif

```

FILE OTAUpdate.h

```

#ifndef OTA_UPDATE_H
#define OTA_UPDATE_H

#include <WiFi.h>
#include <ArduinoJson.h>
#include "config.h"

// Sử dụng thư viện gốc của ESP32
#include "esp_http_client.h"
#include "esp_https_ota.h"
#include "esp_crt_bundle.h"

unsigned long lastUpdateCheck = 0;

String fetchLatestVersion() {
    String version = "";

    // Khởi tạo config bằng 0 để tránh lỗi thứ tự (Designator Order)
    esp_http_client_config_t config = {};

    config.url = VERSION_URL;
    config.event_handler = NULL;
    config.crt_bundle_attach = esp_crt_bundle_attach;
    config.timeout_ms = 5000;

    esp_http_client_handle_t client =
    esp_http_client_init(&config);

    esp_err_t err = esp_http_client_open(client, 0);

    if (err == ESP_OK) {
        esp_http_client_fetch_headers(client);
    }
}

```

```

                                int                content_length                =
esp_http_client_get_content_length(client);

    if (content_length > 0) {
        char *buffer = (char *)malloc(content_length + 1);

        if (buffer) {
            int read_len = esp_http_client_read(client, buffer,
content_length);

            if (read_len > 0) {
                buffer[read_len] = 0;

                version = String(buffer);

                version.trim();
            }

            free(buffer);
        }
    }

    } else {

        Serial.printf("⚠️  Lỗi  HTTP  Check  Version:  %s\n",
esp_err_to_name(err));

    }

    esp_http_client_cleanup(client);

    return version;
}

void downloadAndApplyFirmware() {

    Serial.println("🚀  Bắt đầu cập nhật OTA...");

    esp_http_client_config_t config = {};

    config.url = FIRMWARE_URL;

    config.crt_bundle_attach = esp_crt_bundle_attach;

```

```

config.keep_alive_enable = true;
config.timeout_ms = 10000;
esp_https_ota_config_t ota_config = {};
ota_config.http_config = &config;
esp_err_t ret = esp_https_ota(&ota_config);
if (ret == ESP_OK) {
    Serial.println("✅ Cập nhật xong! Khởi động lại...");
    delay(1000);
    esp_restart();
} else {
    Serial.printf("❌ Lỗi cập nhật: %s\n",
esp_err_to_name(ret));
}
}

void checkForFirmwareUpdate() {
    if (WiFi.status() != WL_CONNECTED) return;
    Serial.println("🔍 Checking firmware...");
    String latestVersion = fetchLatestVersion();
    if (latestVersion.isEmpty()) return;
    Serial.println("💎 Current: " + String(CURRENT_VERSION));
    Serial.println("💎 Latest: " + latestVersion);
    if (latestVersion != CURRENT_VERSION) {
        Serial.println("💡 New version found! Updating...");
        downloadAndApplyFirmware();
    } else {
        Serial.println("✅ Firmware is up to date.");
    }
}

```

```

    }
}

void initialOtaCheck() {
    delay(2000);
    checkForFirmwareUpdate();
}

void otaLoop() {
    if (WiFi.status() == WL_CONNECTED) {
        if (millis() - lastUpdateCheck > OTA_UPDATE_INTERVAL) {
            lastUpdateCheck = millis();
            checkForFirmwareUpdate();
        }
    }
}

#endif

```